HACKER WOMEN COUNCIL
GLOBAL ALLIANCE FOR WOMEN IN CYBERSECURITY

MÁS MUJERES
EN CIBERSEGURIDAD
UN MUNDO MÁS SEGURO

ENTREGABLE 3 & 4

# SQL & WEB SECURITY CHALLENGE

**Parte 1 — SQL: documentación de las 31 consultas. · Parte 2 — OWASP ZAP: primera evaluación de seguridad web sobre Gin & Juice Shop.**

EXPLORA ANALIZA DOCUMENTA APRENDE

Alumna: **Yesica Pérez**

Guía: 02 – SQL & Web Security Challenge

Fecha: 26 de octubre de 2026

TECH · PEOPLE · COMMUNITY
HACKER WOMEN COUNCIL

# SQL Easy – Documentación de las 31 consultas

Oct 24, 2026 · Yesica Perez

## Datos de la actividad

| Campo | Detalle |
|---|---|
| Curso | Hacker Women Council – Ethical Hacking |
| Guía | 02 – SQL & Web Security Challenge |
| Parte | SQL Easy: documentación de las 31 consultas |
| Plataforma | www.sql-easy.com/es |
| Alumna | Yesica Pérez |

**Objetivo:** completar y documentar los 31 ejercicios de SQL Easy en español, explicando cada consulta, sus resultados y su aplicación en el mundo real.

## Cómo completar cada ficha

Cada ejercicio usa los mismos campos de la guía. Completalos apenas ejecutás la consulta.

| N.° | Campo | Qué poner |
|---|---|---|
| 01 | Número y nombre del ejercicio | Ej.: Ejercicio 1: Seleccionar registros |
| 02 | Objetivo del ejercicio | ¿Qué necesitas obtener? |
| 03 | Concepto SQL | Qué instrucción o tema se practica |
| 04 | Consulta ejecutada | El código SQL que escribiste |
| 05 | Resultado | Captura del resultado en SQL Easy |
| 06 | Interpretación | Qué información devuelve la consulta |
| 07 | Aprendizaje | Qué comprendiste o qué dificultad resolviste |
| 08 | Aplicación en el mundo real | Cómo podrías usar esta consulta en una organización |

- Redactá con tus propias palabras, sobre todo el campo Aprendizaje.
- Si una consulta no funcionó de entrada, explicá en Aprendizaje qué tuviste que corregir.
- Usá las pistas solo como apoyo y anotá si las necesitaste.
- Entrega final en PDF, de 15 a 25 páginas aprox., con capturas claras y legibles.

## Ejercicios

### Ejercicio 1 – SELECT: seleccionar todos los datos

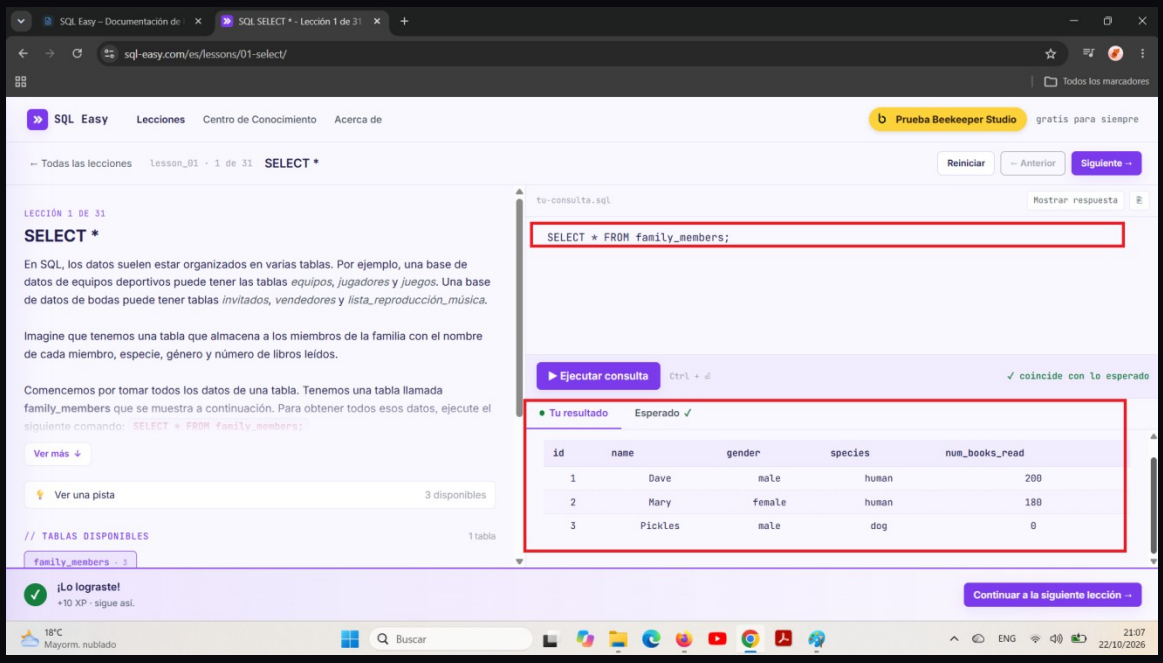
**Objetivo:** mostrar todos los registros y columnas de la tabla family_members.

**Concepto SQL:** `SELECT *` devuelve todas las columnas de una tabla; `FROM` indica de qué tabla leer.

**Consulta ejecutada:**

```
SELECT * FROM family_members;
```

**Resultado:**




Devuelve la tabla family_members completa: todas sus filas con todas sus columnas.

**Interpretación:** se ve el contenido completo de la tabla, sin filtros ni orden.

**Aprendizaje:** aprendí que `SELECT *` es la forma más rápida de ver una tabla completa y entender su estructura antes de armar consultas más específicas.

**Aplicación real:** revisar rápido una tabla nueva. En seguridad, es lo que busca un atacante con una inyección SQL: volcar tablas enteras.

### Ejercicio 2 – SELECT de columnas específicas

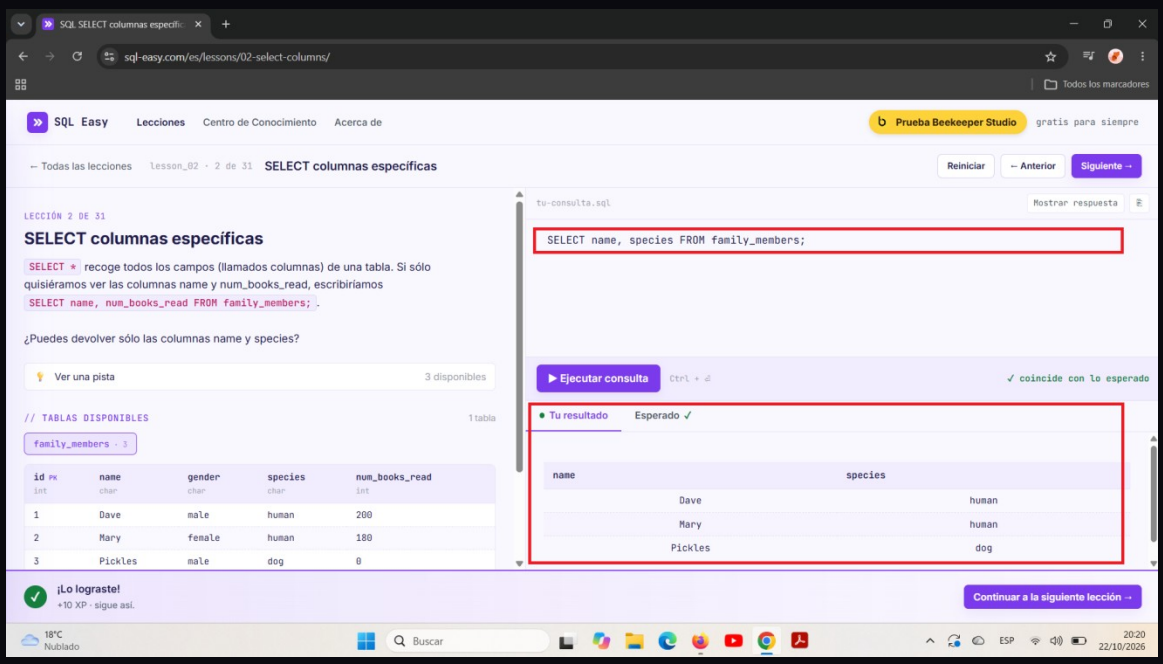
**Objetivo:** mostrar solo el nombre y la especie de cada integrante de family_members.

**Concepto SQL:** listar columnas después de `SELECT`, separadas por coma, limita la salida a esos campos.

**Consulta ejecutada:**

```
SELECT name, species FROM family_members;
```

**Resultado:**




Devuelve una fila por integrante, pero solo con las columnas name y species.

**Interpretación:** se obtiene la misma cantidad de filas que en el ejercicio 1, pero con menos columnas: solo la información pedida.

**Aprendizaje:** comprendí que puedo elegir qué columnas mostrar y que el orden en que las escribo es el orden en que aparecen en el resultado.

**Aplicación real:** pedir solo los campos necesarios mejora el rendimiento y aplica el principio de mínimo privilegio: no exponer columnas sensibles (contraseñas, DNI) si no hacen falta.

## Ejercicio 3 – WHERE: filtrar por coincidencia

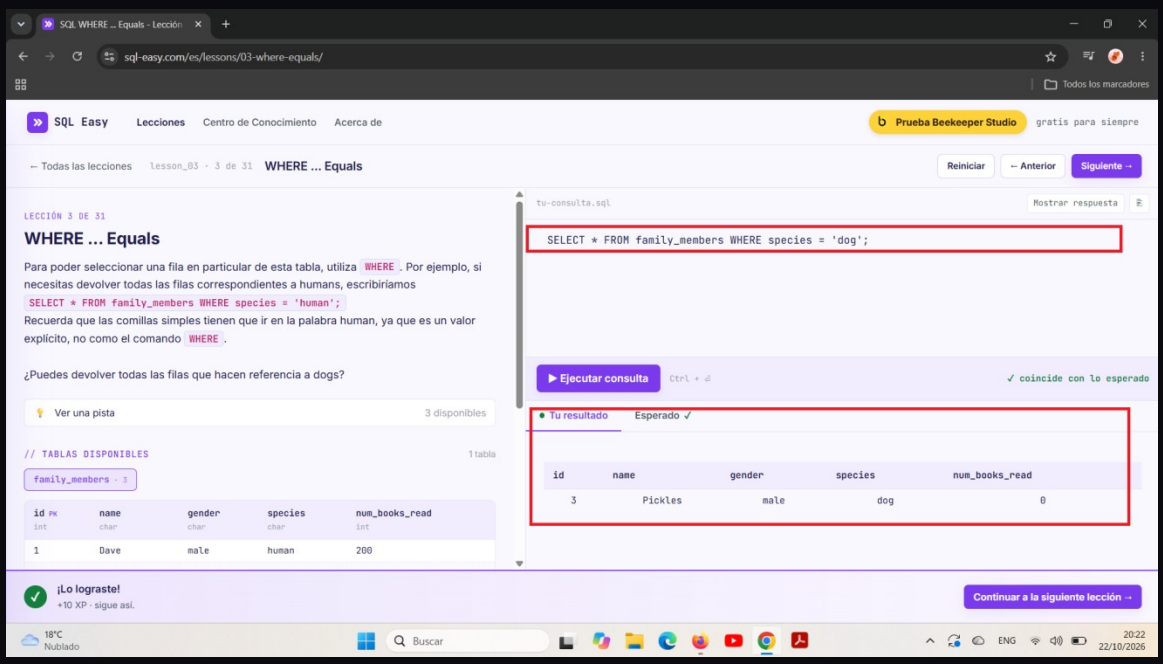
**Objetivo:** mostrar solo los integrantes cuya especie sea perro.

**Concepto SQL:** `WHERE` filtra filas según una condición; los textos van entre comillas simples ('dog').

**Consulta ejecutada:**

```
SELECT * FROM family_members WHERE species = 'dog';
```

**Resultado:**




Devuelve únicamente las filas de family_members donde species es 'dog'.

**Interpretación:** el filtro descarta las filas que no cumplen la condición; el resultado es un subconjunto de la tabla.

**Aprendizaje:** aprendí que los textos en WHERE van entre comillas simples y que el valor tiene que coincidir exactamente con lo guardado en la tabla.

**Aplicación real:** es la base de cualquier búsqueda (un usuario por su email, un pedido por su número). En seguridad, es la cláusula que se manipula en una inyección SQL: con `' OR '1'='1` la condición siempre es verdadera y el filtro deja de funcionar.

## Ejercicio 4 – WHERE con operadores de comparación (>)

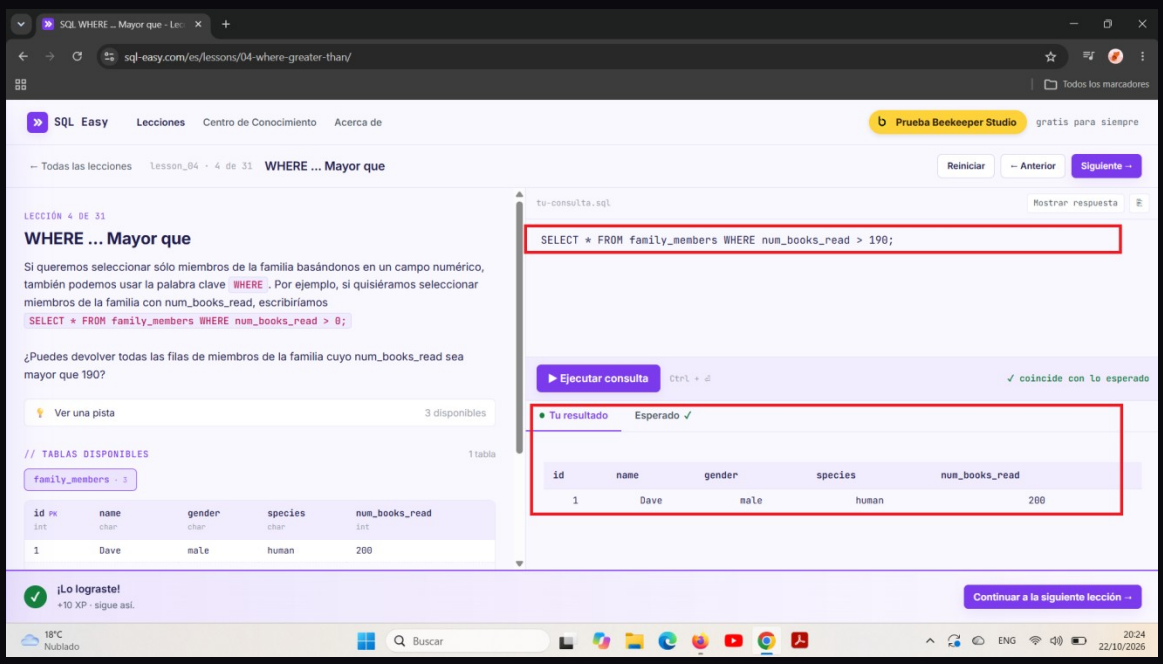
**Objetivo:** encontrar a los integrantes que leyeron más de 190 libros.

**Concepto SQL:** `WHERE` admite operadores numéricos: `>`, `<`, `=`, `!=`.

**Consulta ejecutada:**

```
SELECT * FROM family_members WHERE num_books_read > 190;
```

**Resultado:**




Devuelve solo las filas con num_books_read mayor a 190.

**Interpretación:** el valor 190 queda excluido: `>` es estricto, solo entran los que lo superan.

**Aprendizaje:** comprendí que WHERE también compara números y que en ese caso no hacen falta comillas.

**Aplicación real:** filtrar por umbrales, como clientes con deuda mayor a cierto monto o logins fallidos por encima de un límite (indicio de fuerza bruta).

## Ejercicio 5 – WHERE con mayor o igual (>=)

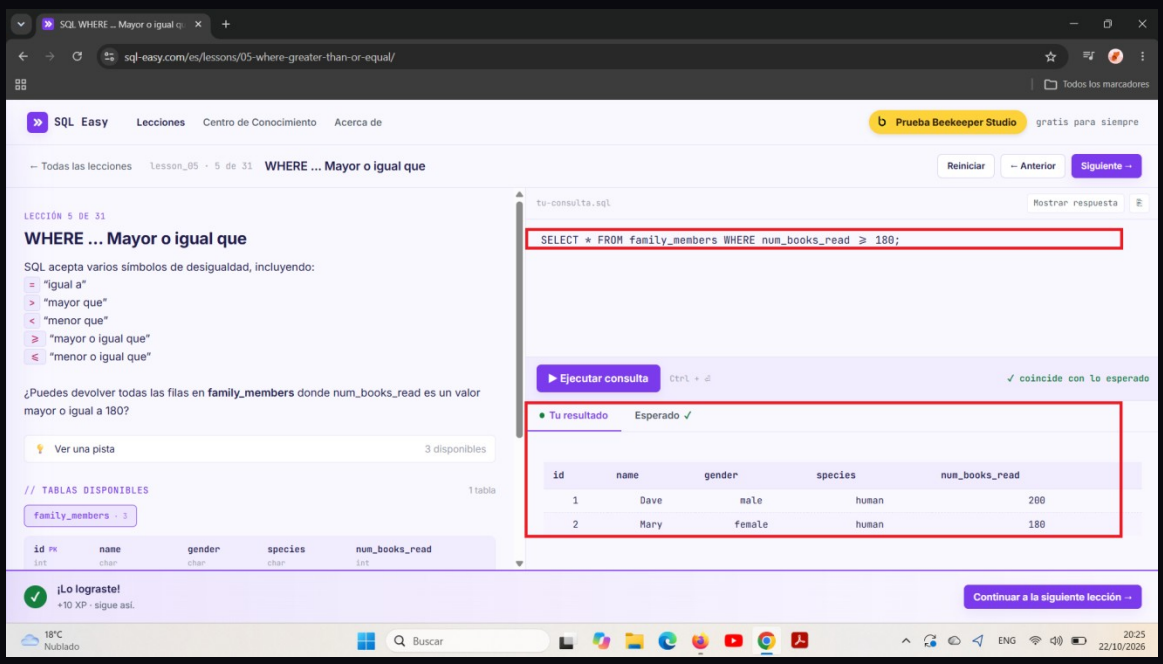
**Objetivo:** encontrar a los integrantes que leyeron 180 libros o más.

**Concepto SQL:** `>=` incluye el valor límite, a diferencia de `>`.

**Consulta ejecutada:**

```
SELECT * FROM family_members WHERE num_books_read >= 180;
```

**Resultado:**




Devuelve las filas con num_books_read igual o mayor a 180.

**Interpretación:** comparado con el ejercicio anterior, ahora también entran los registros que coinciden exactamente con el límite.

**Aprendizaje:** entendí la diferencia entre `>` y `>=`: un solo carácter define si el valor límite queda incluido o no.

**Aplicación real:** definir bien si el límite se incluye evita errores de negocio, como dejar afuera a quien cumple justo el requisito. En QA es el caso típico de prueba de valores límite.

## Ejercicio 6 – AND: combinar condiciones

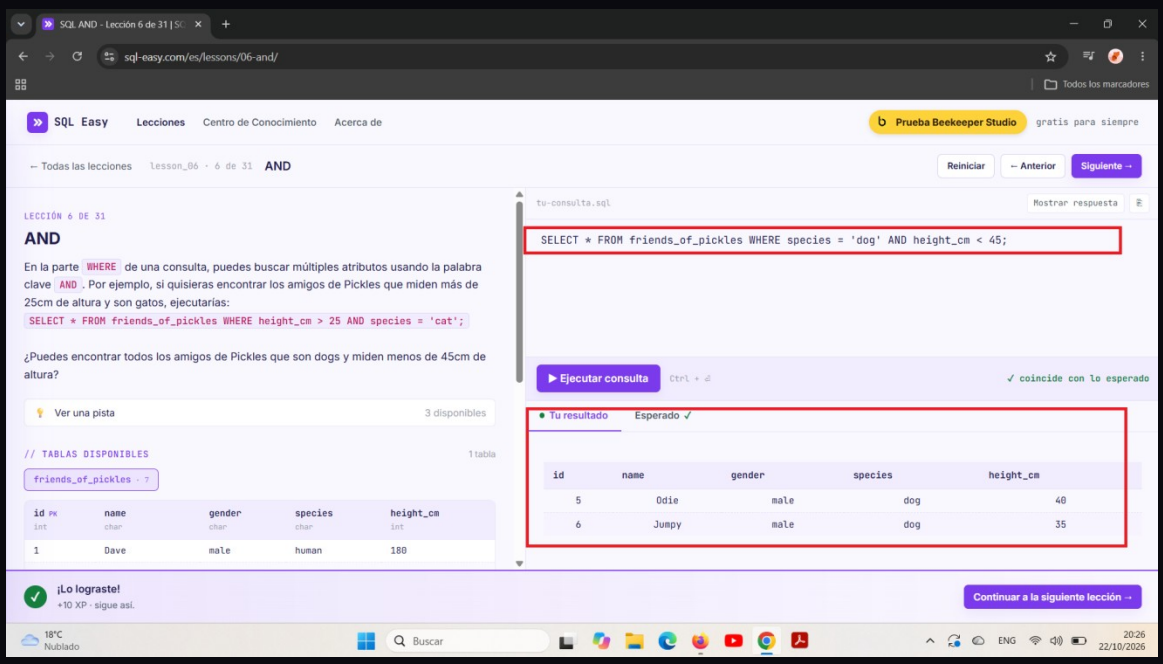
**Objetivo:** mostrar los perros de friends_of_pickles que miden menos de 45 cm.

**Concepto SQL:** `AND` exige que se cumplan todas las condiciones a la vez.

**Consulta ejecutada:**

```
SELECT * FROM friends_of_pickles WHERE species = 'dog' AND height_cm < 45;
```

**Resultado:**




Devuelve solo las filas que son 'dog' y además tienen height_cm menor a 45.

**Interpretación:** cada condición que se suma con AND achica el resultado: es más restrictivo.

**Aprendizaje:** aprendí a combinar condiciones con AND y que cada condición nueva achica el resultado.

**Aplicación real:** búsquedas con varios criterios, como transacciones de un mismo usuario por encima de cierto monto, típicas en detección de fraude.

## Ejercicio 7 – OR: condiciones alternativas

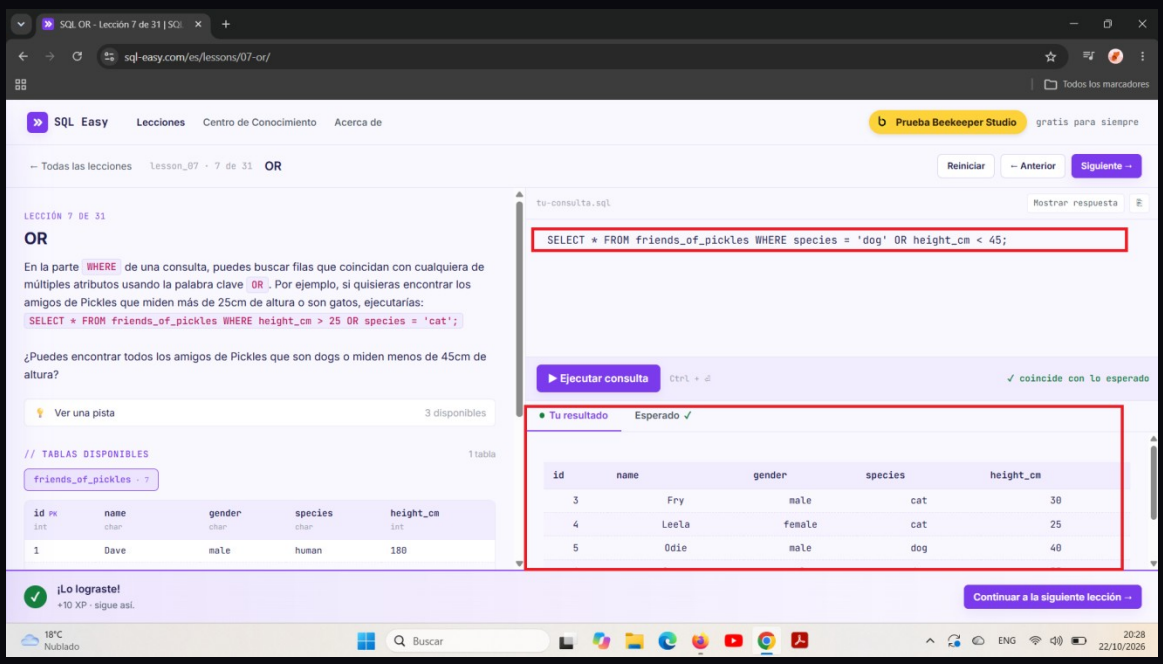
**Objetivo:** mostrar los registros que son perros o que miden menos de 45 cm.

**Concepto SQL:** `OR` devuelve la fila si se cumple al menos una de las condiciones.

**Consulta ejecutada:**

```
SELECT * FROM friends_of_pickles WHERE species = 'dog' OR height_cm < 45;
```

**Resultado:**




Devuelve los perros de cualquier altura y, además, cualquier animal de menos de 45 cm.

**Interpretación:** a diferencia de AND, OR amplía el resultado; por eso devuelve igual o más filas que el ejercicio 6.

**Aprendizaje:** comprendí que OR funciona al revés que AND: amplía el resultado. Comparar los ejercicios 6 y 7 me ayudó a verlo claramente.

**Aplicación real:** buscar varios casos a la vez (estado 'pendiente' o 'rechazado'). Es también el operador del clásico `' OR '1'='1` en inyecciones SQL, que vuelve verdadera cualquier condición.

## Ejercicio 8 – IN / NOT IN: listas de valores

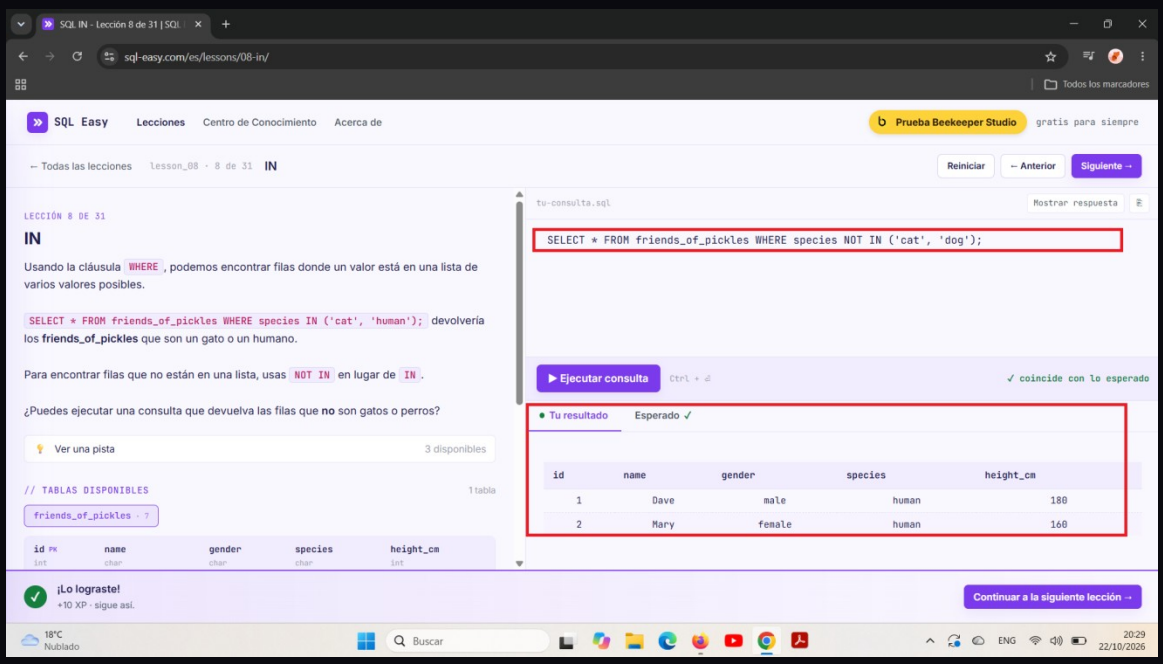
**Objetivo:** mostrar los registros cuya especie no sea gato ni perro.

**Concepto SQL:** `IN` compara contra una lista de valores; `NOT IN` excluye los que están en la lista.

**Consulta ejecutada:**

```
SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');
```

**Resultado:**




Devuelve las filas de friends_of_pickles cuya especie es distinta de 'cat' y de 'dog'.

**Interpretación:** es una forma más corta y legible de escribir `species != 'cat' AND species != 'dog'`.

**Aprendizaje:** aprendí que NOT IN reemplaza varias condiciones con `!=` y hace la consulta más corta y fácil de leer.

**Aplicación real:** listas permitidas o de exclusión (allowlist / denylist), por ejemplo IPs bloqueadas o roles autorizados.

## Ejercicio 9 – DISTINCT: valores únicos

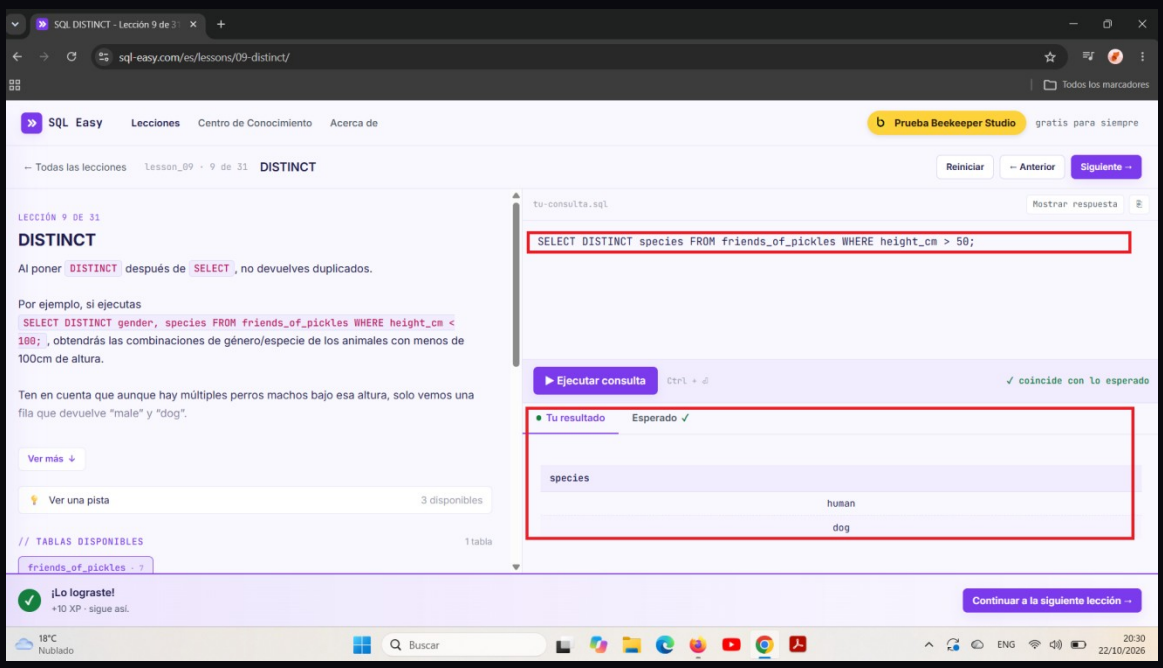
**Objetivo:** obtener las especies, sin repetir, de los animales que miden más de 50 cm.

**Concepto SQL:** `DISTINCT` elimina las filas duplicadas del resultado.

**Consulta ejecutada:**

```
SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;
```

**Resultado:**




Devuelve una lista de especies, cada una, una sola vez.

**Interpretación:** aunque varios animales de la misma especie cumplan la condición, cada especie aparece una sola vez.

**Aprendizaje:** entendí que DISTINCT elimina las repeticiones del resultado, no de la tabla, y que se aplica sobre las columnas del SELECT.

**Aplicación real:** saber qué valores distintos existen en un campo: tipos de eventos en un log, navegadores que acceden a un sitio o países de origen de conexiones.

## Ejercicio 10 – ORDER BY: ordenar resultados

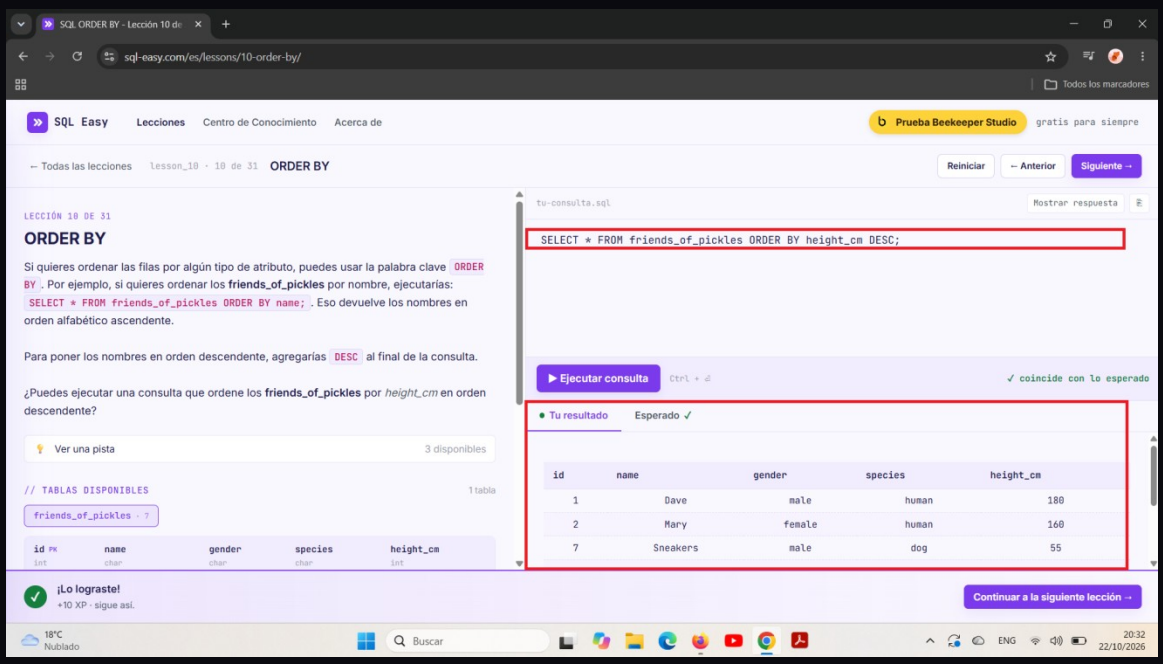
**Objetivo:** listar friends_of_pickles ordenados por altura, de mayor a menor.

**Concepto SQL:** `ORDER BY` ordena por una columna; `DESC` es descendente y `ASC` (por defecto) ascendente.

**Consulta ejecutada:**

```
SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;
```

**Resultado:**




Devuelve todas las filas, con el animal más alto primero.

**Interpretación:** no cambia qué filas aparecen, solo su orden.

**Aprendizaje:** aprendí que ORDER BY ordena de forma ascendente por defecto y que hay que agregar DESC para invertir el orden.

**Aplicación real:** rankings y reportes, como los usuarios con más intentos fallidos primero o los eventos más recientes arriba en un log.

## Ejercicio 11 – LIMIT: limitar filas

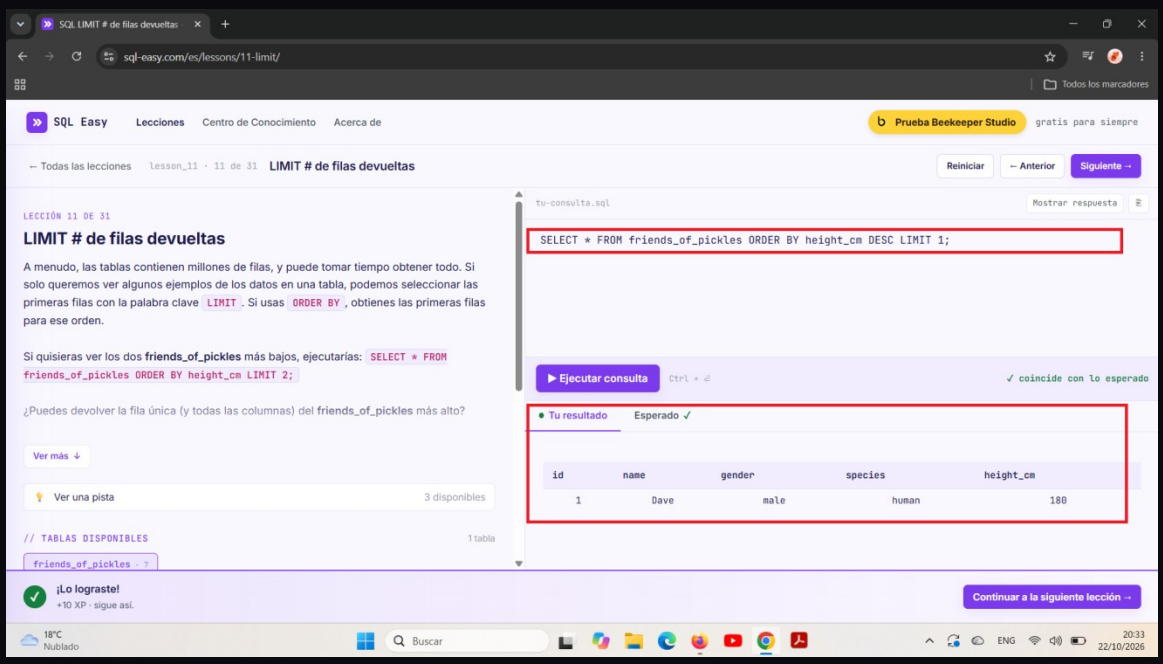
**Objetivo:** obtener solo el animal más alto.

**Concepto SQL:** `LIMIT` restringe la cantidad de filas devueltas; combinado con ORDER BY permite quedarse con el primero o los N primeros.

**Consulta ejecutada:**

```
SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;
```

**Resultado:**




Devuelve una sola fila: el registro con mayor height_cm.

**Interpretación:** el orden descendente pone al más alto primero y LIMIT 1 se queda solo con ese.

**Aprendizaje:** comprendí que LIMIT por sí solo no garantiza qué fila obtengo: necesita ORDER BY para que el resultado tenga sentido.

**Aplicación real:** top N (los 10 productos más vendidos) y paginación de resultados. En ataques, LIMIT se usa para extraer datos de a una fila por vez.

## Ejercicio 12 – COUNT: contar filas

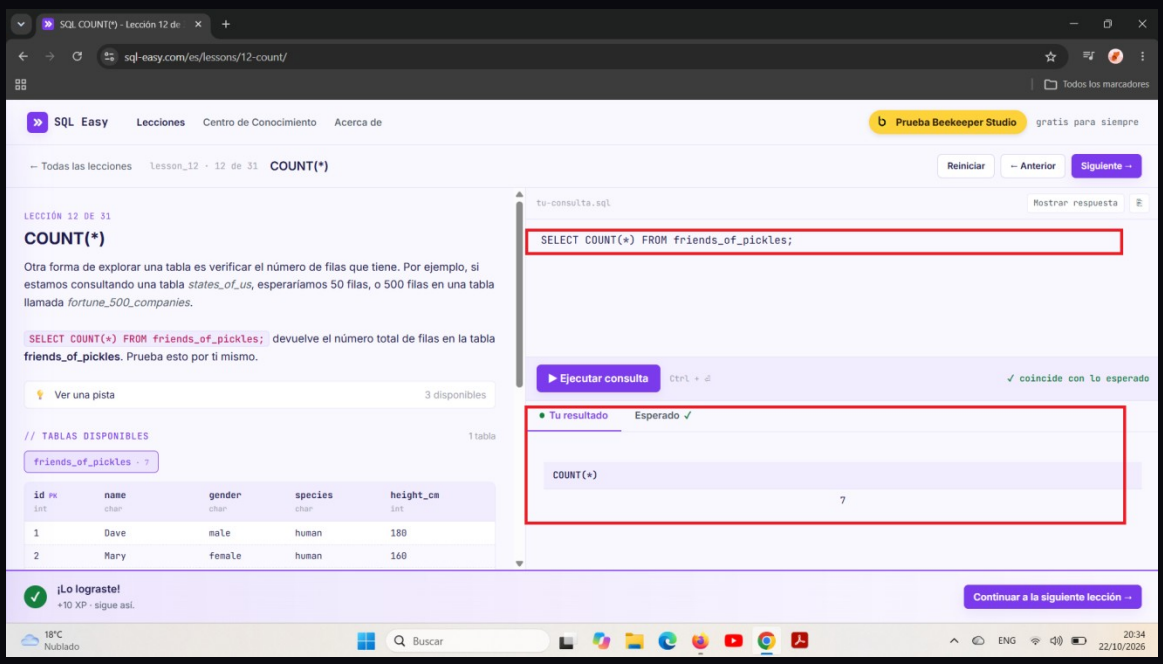
**Objetivo:** contar cuántos registros tiene la tabla friends_of_pickles.

**Concepto SQL:** `COUNT(*)` es una función de agregación que cuenta filas.

**Consulta ejecutada:**

```
SELECT COUNT(*) FROM friends_of_pickles;
```

**Resultado:**




Devuelve un único número: el total de filas de la tabla, que en este caso es **7**.

**Interpretación:** en vez de listar los registros, se obtiene un resumen: cuántos hay.

**Aprendizaje:** aprendí que `COUNT(*)` resume toda la tabla en un solo número en lugar de listar los registros.

**Aplicación real:** métricas rápidas, como la cantidad de usuarios registrados, de alertas del día o de intentos de login.

## Ejercicio 13 – COUNT con WHERE

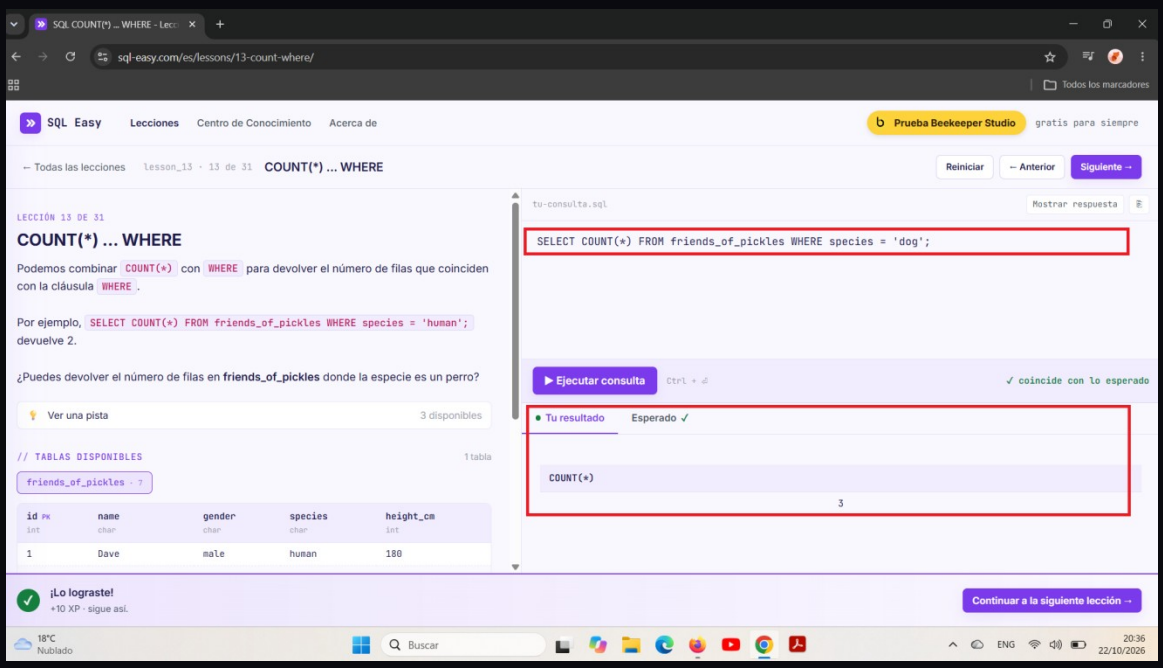
**Objetivo:** contar cuántos perros hay en friends_of_pickles.

**Concepto SQL:** las funciones de agregación se combinan con WHERE: primero se filtra y después se cuenta.

**Consulta ejecutada:**

```
SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';
```

**Resultado:**




Devuelve un número: la cantidad de filas donde species es 'dog'que es **3**.

**Interpretación:** el conteo se aplica solo sobre las filas que pasan el filtro.

**Aprendizaje:** comprendí el orden en que trabaja la consulta: primero filtra con WHERE y después cuenta.

**Aplicación real:** contar eventos de un tipo: logins fallidos de un usuario, transacciones rechazadas o accesos desde una misma IP.

## Ejercicio 14 – SUM: sumar valores

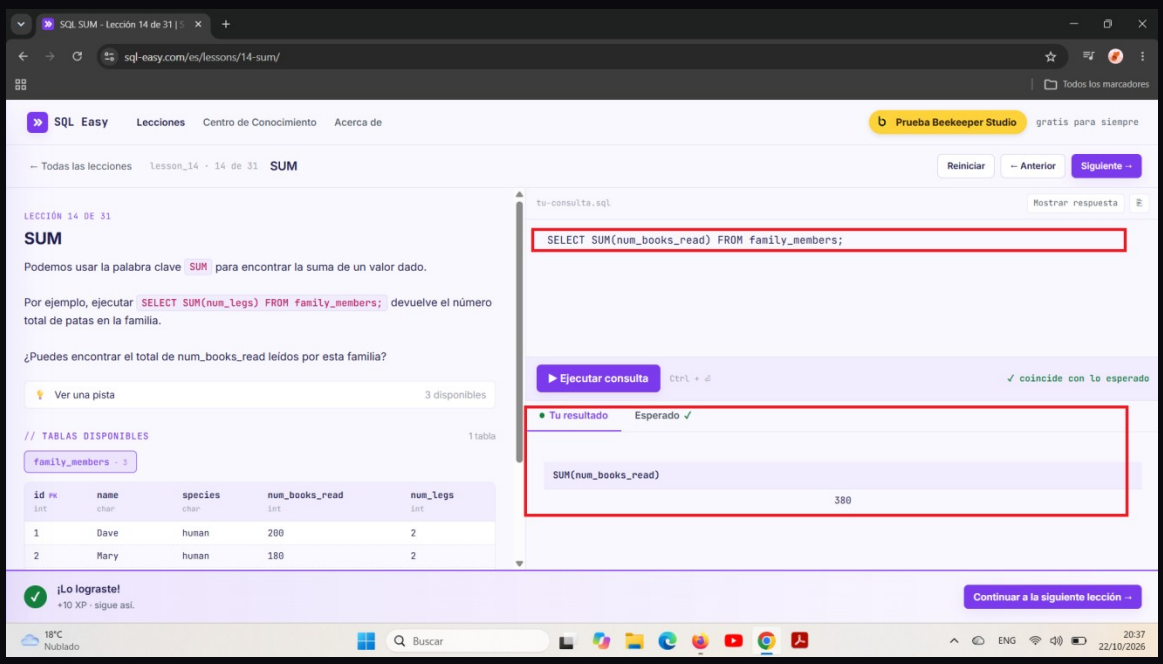
**Objetivo:** calcular el total de libros leídos por toda la familia.

**Concepto SQL:** `SUM()` suma los valores de una columna numérica.

**Consulta ejecutada:**

```
SELECT SUM(num_books_read) FROM family_members;
```

**Resultado:**




Devuelve un único valor con la suma de num_books_read que da **380**.

**Interpretación:** resume toda la columna en un total.

**Aprendizaje:** aprendí que SUM solo tiene sentido con columnas numéricas y que devuelve un único total.

**Aplicación real:** totales de facturación, montos transferidos por cuenta o volumen de datos enviados (útil para detectar exfiltración).

## Ejercicio 15 – AVG: promedio

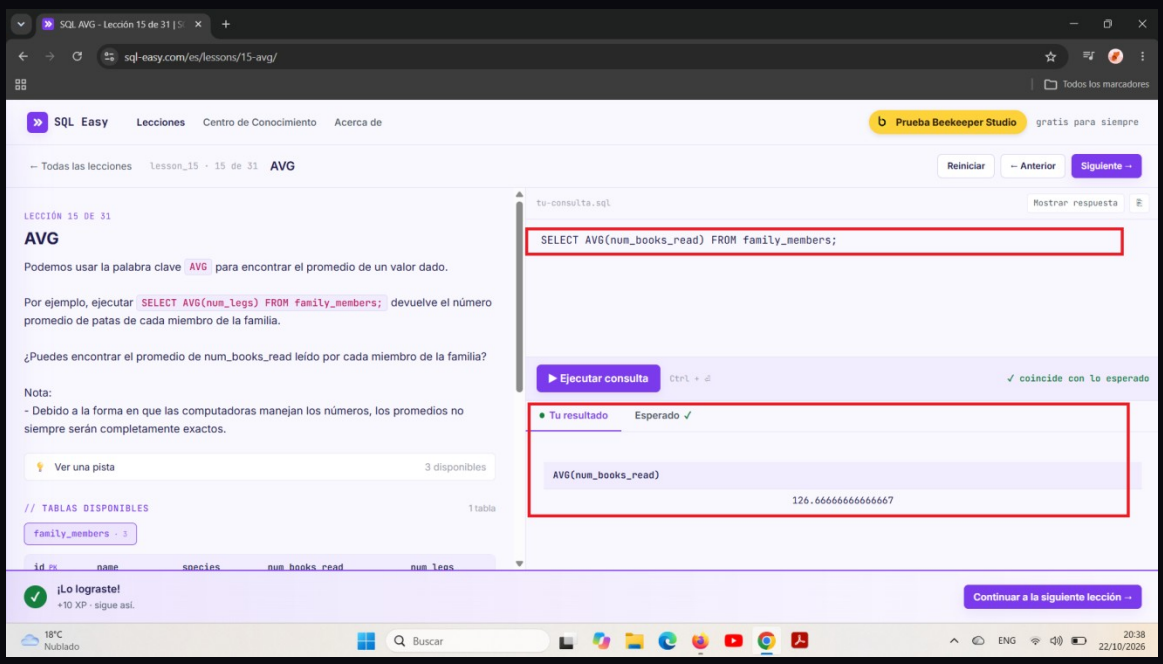
**Objetivo:** calcular el promedio de libros leídos por integrante.

**Concepto SQL:** `AVG()` calcula el promedio de una columna numérica.

**Consulta ejecutada:**

```
SELECT AVG(num_books_read) FROM family_members;
```

**Resultado:**




Devuelve un único valor con el promedio de num_books_read que es **126,67** (126.6666...).

**Interpretación:** indica el valor típico del grupo y puede tener decimales.

**Aprendizaje:** entendí que AVG calcula el promedio directamente, sin tener que sumar y dividir por mi cuenta.

**Aplicación real:** establecer una línea base de comportamiento normal (por ejemplo, el monto promedio por cliente) para detectar desvíos o anomalías.

## Ejercicio 16 – MAX / MIN: máximo y mínimo

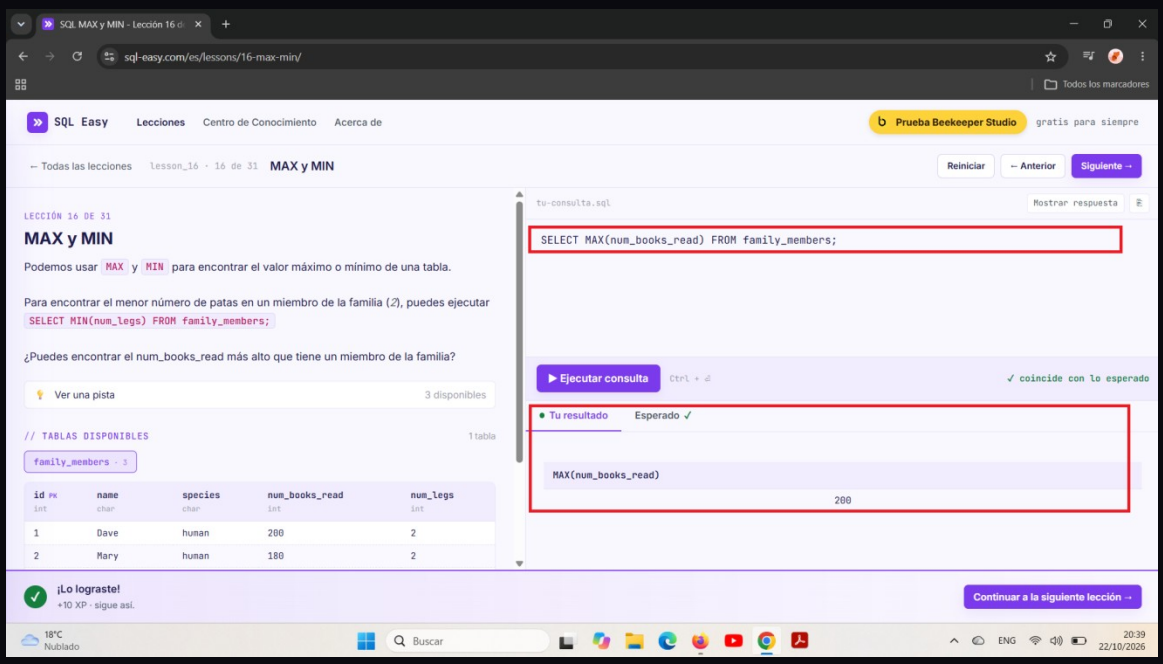
**Objetivo:** encontrar la mayor cantidad de libros leídos por un integrante.

**Concepto SQL:** `MAX()` devuelve el valor más alto de una columna y `MIN()` el más bajo.

**Consulta ejecutada:**

```
SELECT MAX(num_books_read) FROM family_members;
```

**Resultado:**




Devuelve un único valor: el máximo de num_books_read que es **200**.

**Interpretación:** muestra el valor máximo, pero no a quién pertenece; eso se resuelve en el ejercicio 18.

**Aprendizaje:** aprendí que MAX me da el valor más alto, pero no me dice a qué registro pertenece.

**Aplicación real:** detectar extremos, como la transacción más grande del día o el último acceso registrado (MAX sobre fechas).

## Ejercicio 17 – GROUP BY: agrupar

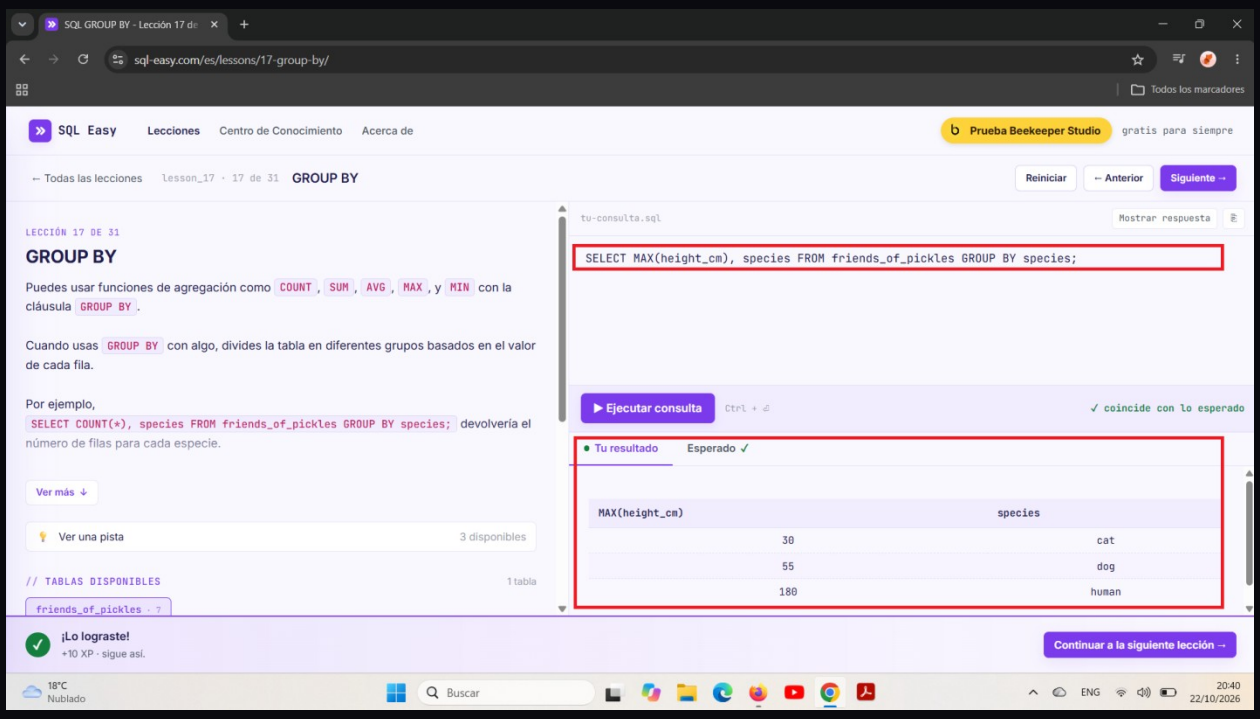
**Objetivo:** obtener la altura máxima de cada especie.

**Concepto SQL:** `GROUP BY` agrupa las filas con el mismo valor y aplica la función de agregación a cada grupo.

**Consulta ejecutada:**

```
SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;
```

**Resultado:**




Devuelve una fila por especie, con su altura máxima.

**Interpretación:** en lugar de un solo máximo general, se obtiene un máximo por grupo.

**Aprendizaje:** comprendí que GROUP BY divide la tabla en grupos y que la función de agregación se calcula por separado para cada uno.

**Aplicación real:** reportes por categoría: intentos de login fallidos por usuario o por IP, ventas por sucursal o alertas por severidad.

## Ejercicio 18 – Consultas anidadas (subconsultas)

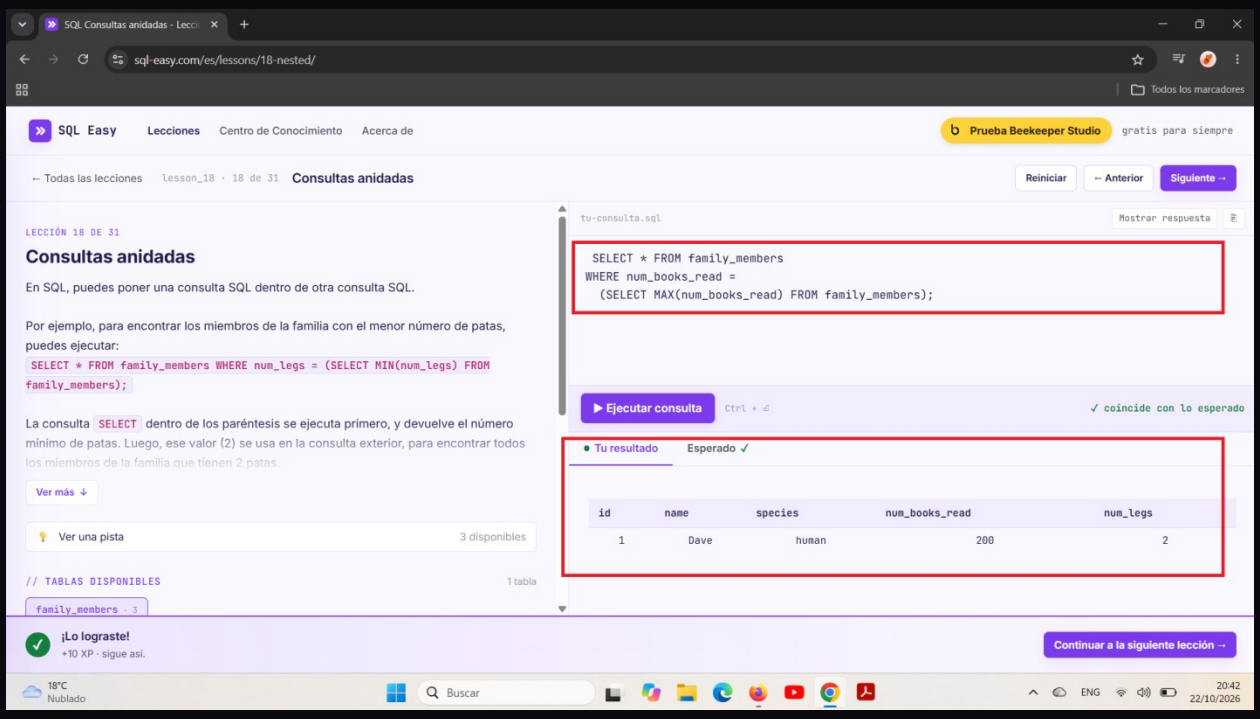
**Objetivo:** mostrar los datos completos del integrante que leyó más libros.

**Concepto SQL:** una subconsulta es un SELECT dentro de otro: la interna calcula el máximo y la externa busca la fila con ese valor.

**Consulta ejecutada:**

```
SELECT * FROM family_members
WHERE num_books_read =
  (SELECT MAX(num_books_read) FROM family_members);
```

**Resultado:**




Devuelve la fila completa del integrante con el mayor num_books_read.

**Interpretación:** resuelve la limitación del ejercicio 16: ahora se sabe quién tiene el máximo, no solo el valor.

**Aprendizaje:** aprendí a usar una consulta dentro de otra; la subconsulta resolvió el problema del ejercicio 16 de saber quién tenía el máximo.

**Aplicación real:** comparar contra un valor calculado (clientes que gastan más que el promedio). Las subconsultas también se usan en inyecciones SQL avanzadas para extraer datos de otras tablas.

## Ejercicio 19 – NULL: valores vacíos

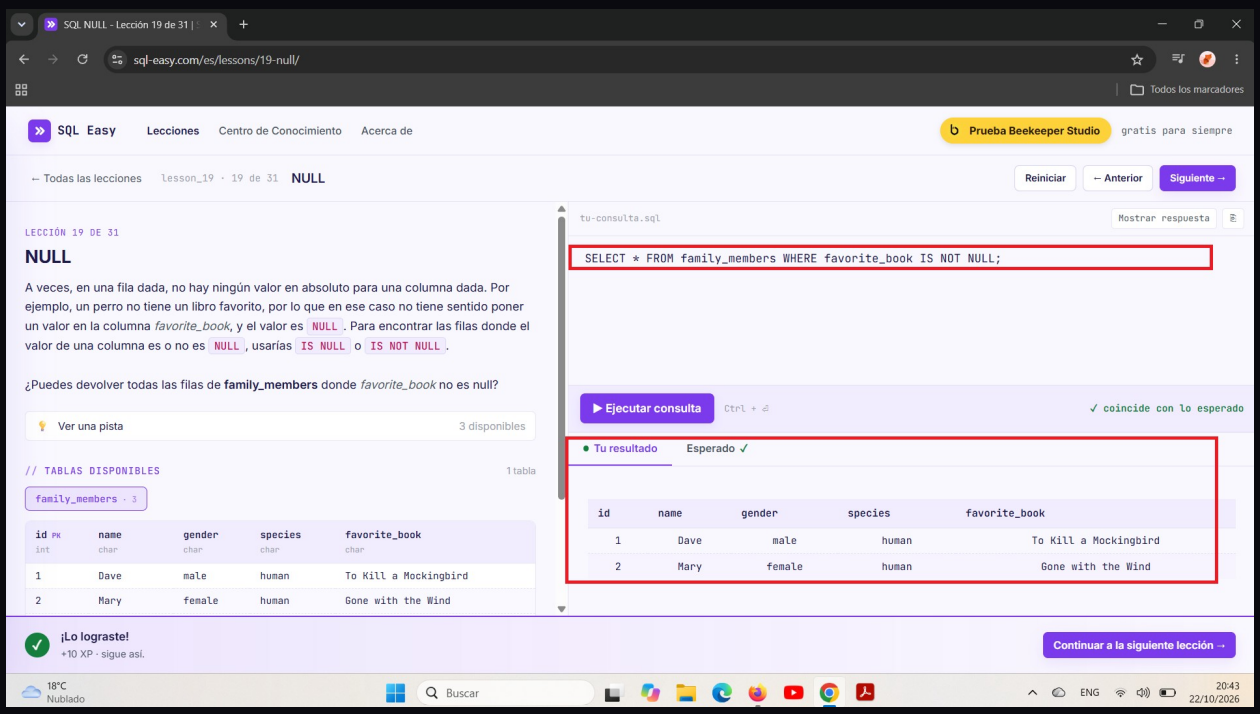
**Objetivo:** mostrar los integrantes que tienen un libro favorito cargado.

**Concepto SQL:** `NULL` representa la ausencia de valor; se consulta con `IS NULL` o `IS NOT NULL`, nunca con `=`.

**Consulta ejecutada:**

```
SELECT * FROM family_members WHERE favorite_book IS NOT NULL;
```

**Resultado:**




Devuelve solo las filas donde favorite_book tiene un valor.

**Interpretación:** los registros sin libro favorito quedan afuera; `= NULL` no funcionaría porque NULL no es igual a nada.

**Aprendizaje:** aprendí que NULL no se compara con `=`, sino con IS NULL o IS NOT NULL, porque representa la ausencia de un valor.

**Aplicación real:** control de calidad de datos: detectar registros incompletos, como usuarios sin email verificado o cuentas sin segundo factor configurado.

## Ejercicio 20 – Fechas

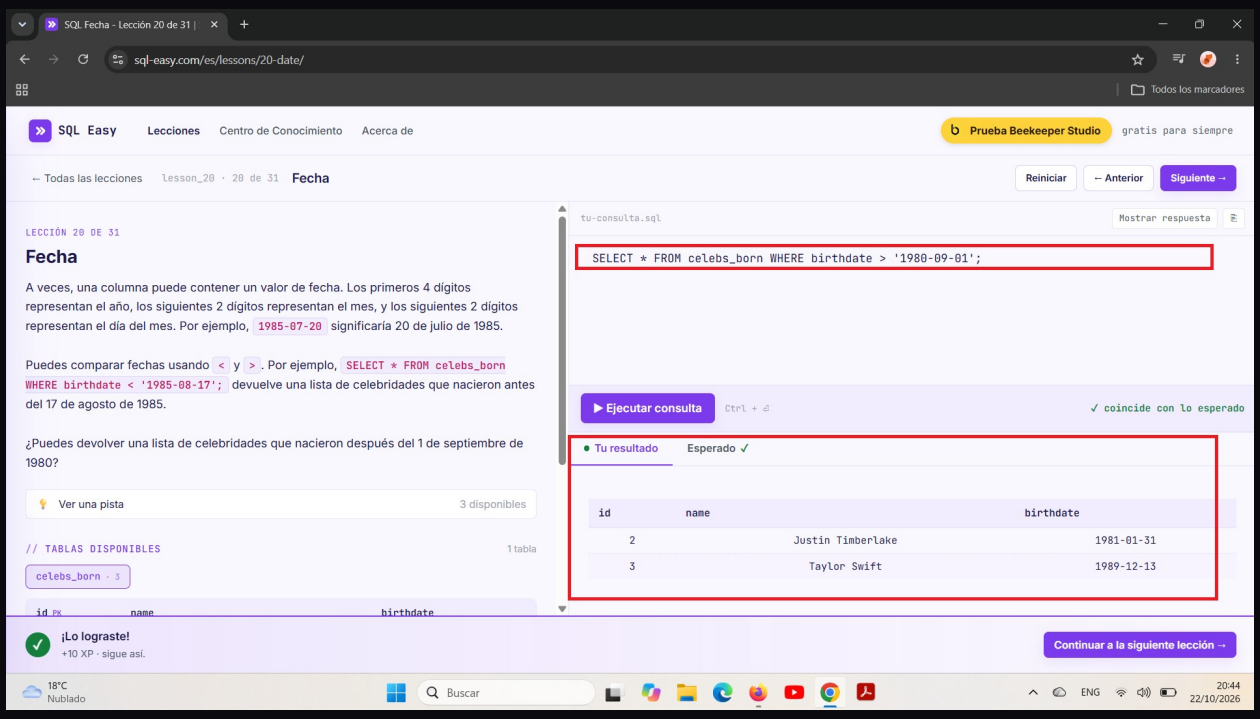
**Objetivo:** mostrar las celebridades nacidas después del 1 de septiembre de 1980.

**Concepto SQL:** las fechas en formato 'AAAA-MM-DD' se comparan con los operadores habituales.

**Consulta ejecutada:**

```
SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';
```

**Resultado:**




Devuelve las filas de celebs_born con birthdate posterior a 1980-09-01.

**Interpretación:** el formato año-mes-día hace que la comparación respete el orden cronológico.

**Aprendizaje:** comprendí que las fechas en formato AAAA-MM-DD se comparan con los mismos operadores que los números.

**Aplicación real:** filtrar por rango de tiempo es central en el análisis forense y de logs: qué pasó antes y después de un incidente.

## Ejercicio 21 – INNER JOIN: unir tablas

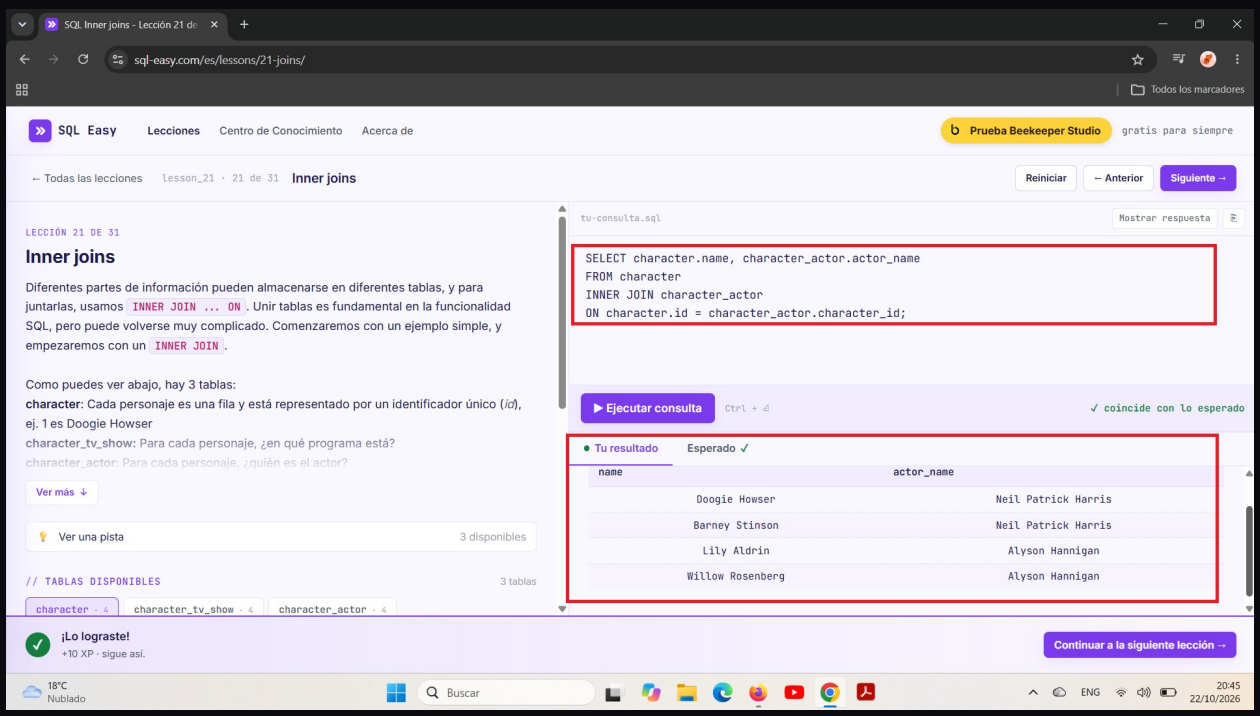
**Objetivo:** mostrar cada personaje junto al nombre del actor que lo interpreta.

**Concepto SQL:** `INNER JOIN` combina las filas de dos tablas que coinciden según la condición del `ON`.

**Consulta ejecutada:**

```
SELECT character.name, character_actor.actor_name
FROM character
INNER JOIN character_actor
ON character.id = character_actor.character_id;
```

**Resultado:**




Devuelve pares personaje–actor para los personajes que tienen coincidencia en character_actor.

**Interpretación:** solo aparecen los personajes con un registro relacionado; los que no tienen actor quedan afuera.

**Aprendizaje:** aprendí cómo se relacionan dos tablas a través de un campo en común y cómo INNER JOIN las combina con ON.

**Aplicación real:** las bases de datos reales separan la información en tablas relacionadas (clientes y pedidos, usuarios y roles) y JOIN las vuelve a unir.

## Ejercicio 22 – Múltiples JOIN

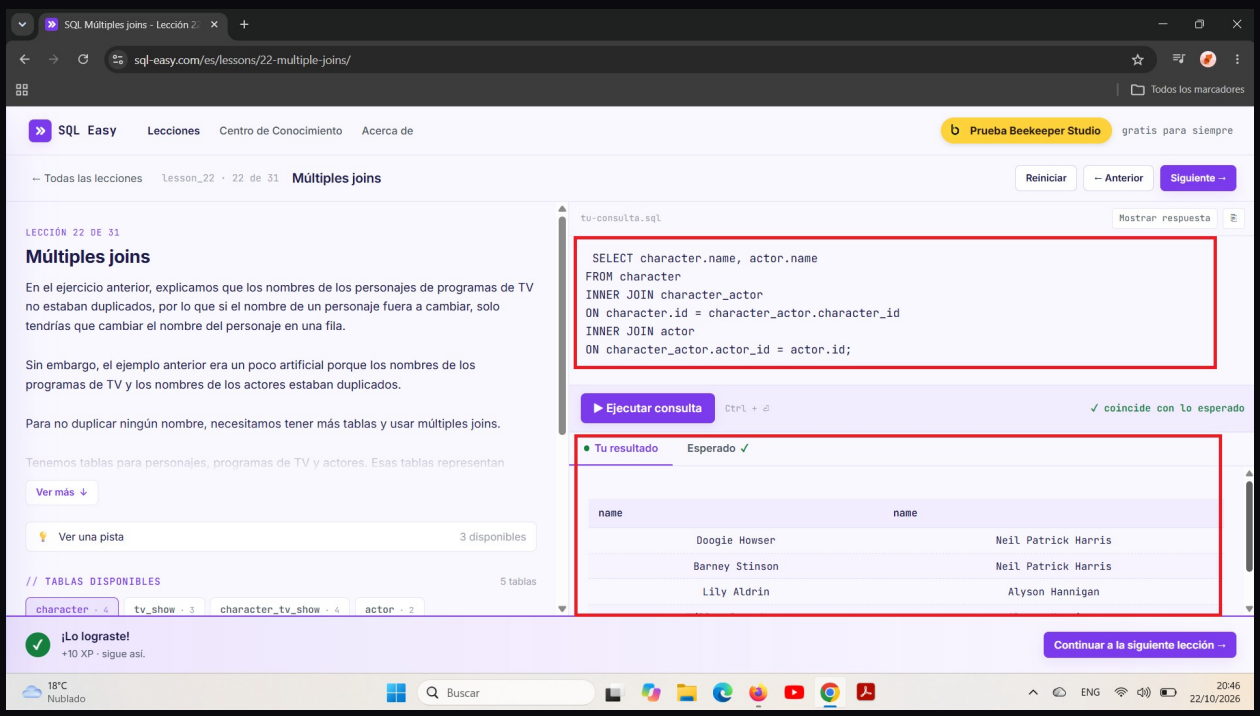
**Objetivo:** mostrar el nombre de cada personaje y el de su actor usando tres tablas.

**Concepto SQL:** se pueden encadenar varios JOIN; character_actor funciona como tabla intermedia entre character y actor.

**Consulta ejecutada:**

```
SELECT character.name, actor.name
FROM character
INNER JOIN character_actor
ON character.id = character_actor.character_id
INNER JOIN actor
ON character_actor.actor_id = actor.id;
```

**Resultado:**




Devuelve pares personaje–actor obtenidos a través de la tabla intermedia.

**Interpretación:** la tabla intermedia resuelve una relación muchos a muchos: un actor puede tener varios personajes y viceversa.

**Aprendizaje:** comprendí el rol de una tabla intermedia y cómo encadenar dos JOIN para llegar de una tabla a otra.

**Aplicación real:** relaciones como usuarios–roles–permisos. Saber qué permisos tiene realmente un usuario es clave en una auditoría de accesos.

## Ejercicio 23 – JOIN con WHERE

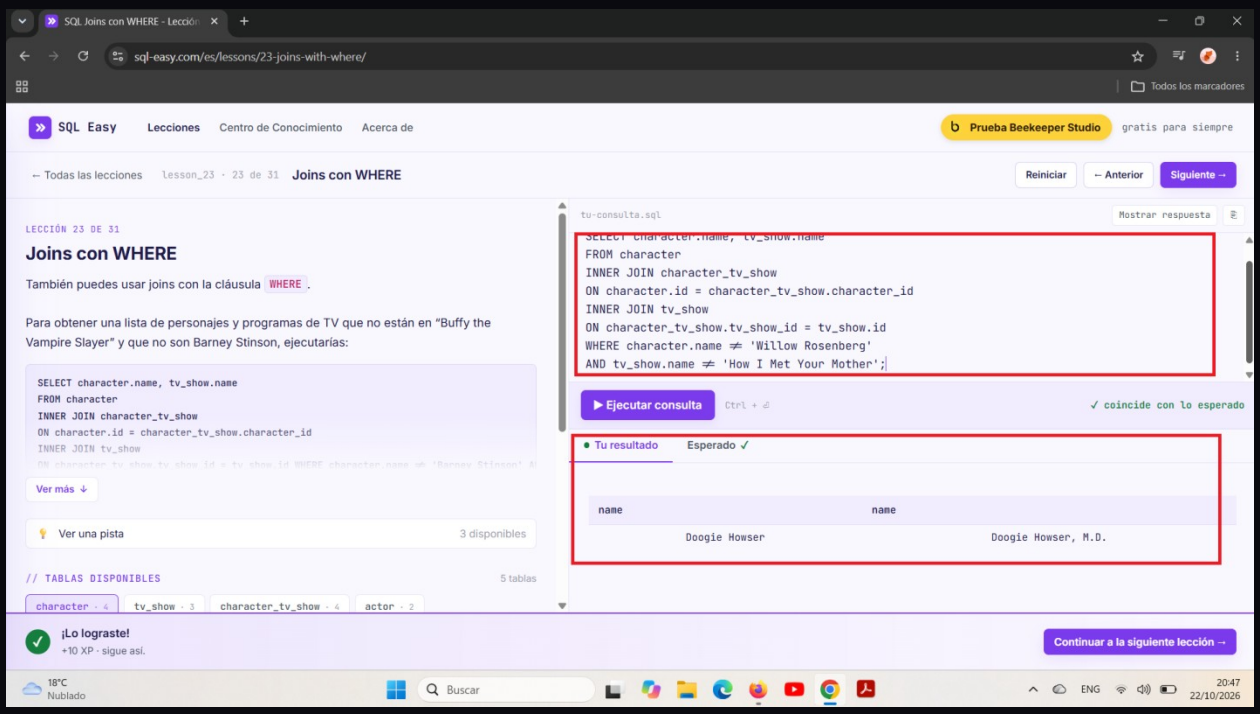
**Objetivo:** listar personajes con sus series, excluyendo a Willow Rosenberg y a How I Met Your Mother.

**Concepto SQL:** después de unir tablas se puede filtrar el resultado con WHERE; `!=` significa distinto.

**Consulta ejecutada:**

```
SELECT character.name, tv_show.name
FROM character
INNER JOIN character_tv_show
ON character.id = character_tv_show.character_id
INNER JOIN tv_show
ON character_tv_show.tv_show_id = tv_show.id
WHERE character.name != 'Willow Rosenberg'
AND tv_show.name != 'How I Met Your Mother';
```

**Resultado:**




Devuelve pares personaje–serie, sin las filas de Willow Rosenberg ni de How I Met Your Mother.

**Interpretación:** primero los JOIN arman la combinación y después el WHERE descarta lo que no interesa.

**Aprendizaje:** aprendí que WHERE se puede usar después de los JOIN para filtrar el resultado ya combinado.

**Aplicación real:** reportes cruzados con exclusiones, por ejemplo accesos a sistemas críticos sin contar las cuentas de servicio.

## Ejercicio 24 – LEFT JOIN

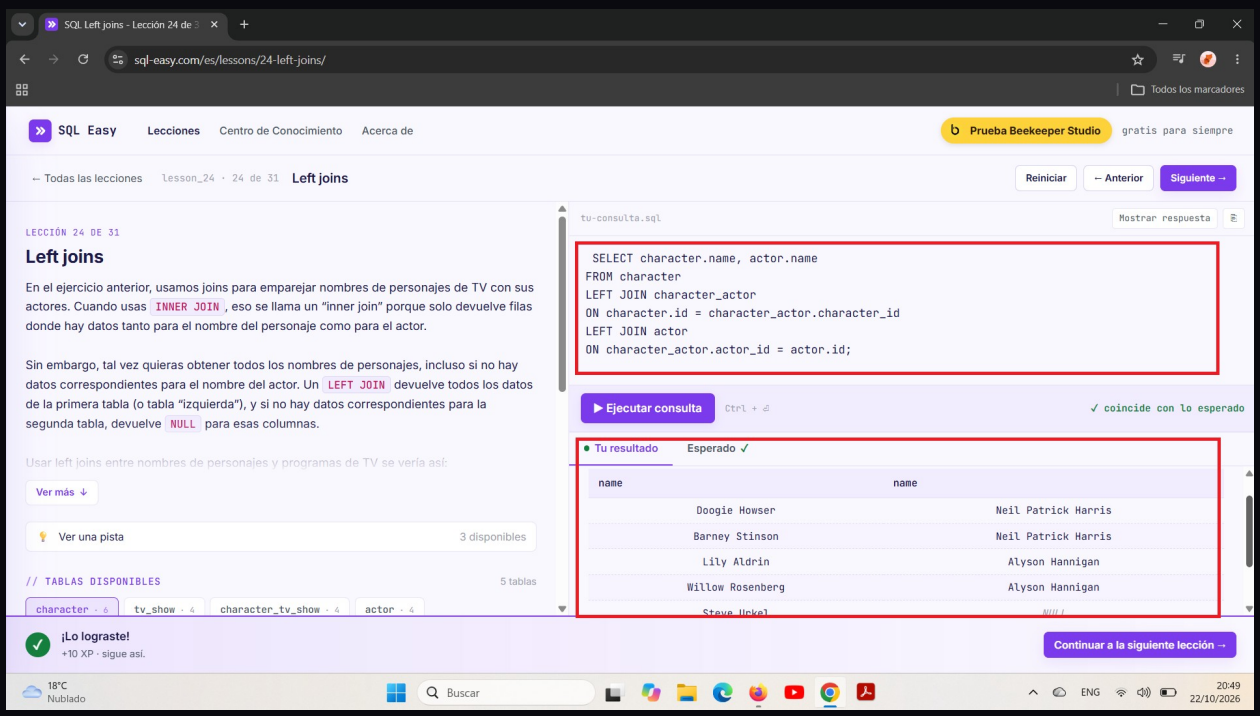
**Objetivo:** mostrar todos los personajes con su actor, aunque no tengan uno asignado.

**Concepto SQL:** `LEFT JOIN` conserva todas las filas de la tabla de la izquierda; si no hay coincidencia, completa con NULL.

**Consulta ejecutada:**

```
SELECT character.name, actor.name
FROM character
LEFT JOIN character_actor
ON character.id = character_actor.character_id
LEFT JOIN actor
ON character_actor.actor_id = actor.id;
```

**Resultado:**




Devuelve todos los personajes; los que no tienen actor aparecen con NULL en esa columna.

**Interpretación:** a diferencia del INNER JOIN del ejercicio 22, no se pierde ningún personaje.

**Aprendizaje:** entendí la diferencia clave entre INNER JOIN y LEFT JOIN: el LEFT conserva todas las filas de la tabla principal aunque no tengan coincidencia.

**Aplicación real:** encontrar lo que falta: usuarios sin rol asignado, activos sin responsable o cuentas sin actividad. Muy útil en auditorías.

## Ejercicio 25 – Alias de tabla

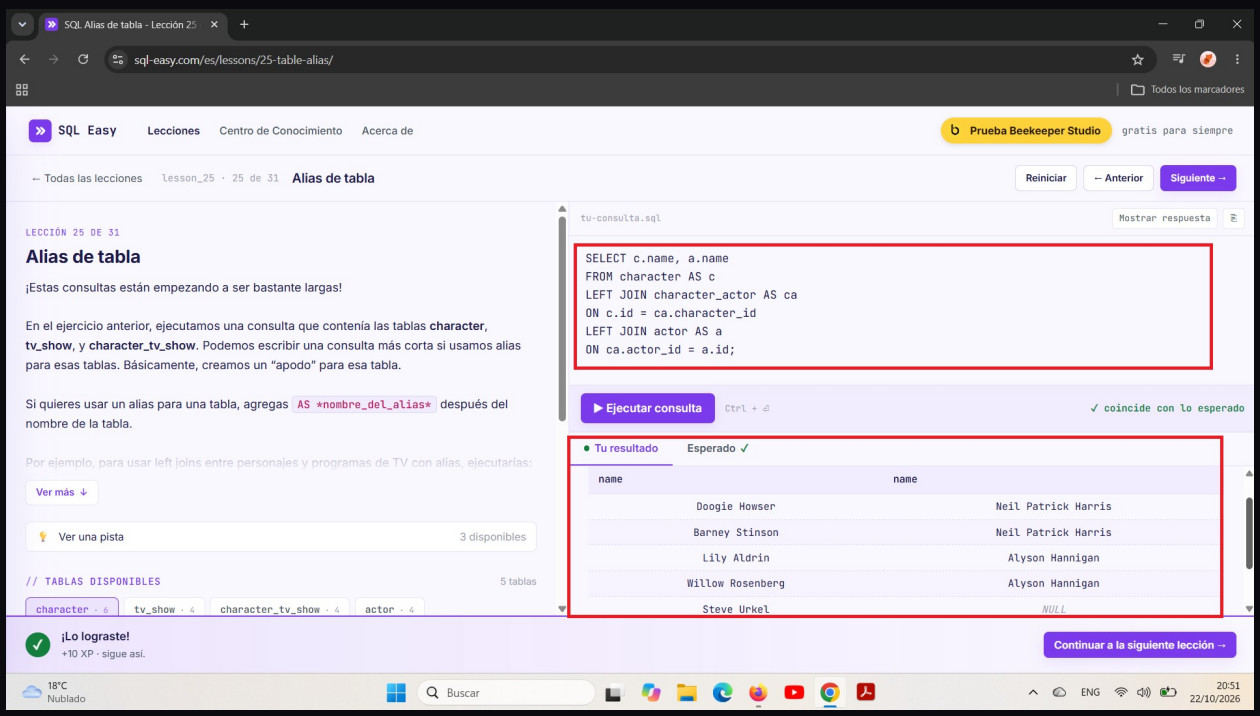
**Objetivo:** repetir la consulta anterior usando nombres cortos para las tablas.

**Concepto SQL:** `AS` asigna un alias a una tabla (c, ca, a) para escribir consultas más cortas.

**Consulta ejecutada:**

```
SELECT c.name, a.name
FROM character AS c
LEFT JOIN character_actor AS ca
ON c.id = ca.character_id
LEFT JOIN actor AS a
ON ca.actor_id = a.id;
```

**Resultado:**




Devuelve el mismo resultado que el ejercicio 24.

**Interpretación:** el alias no cambia los datos, solo hace la consulta más legible y fácil de mantener.

**Aprendizaje:** aprendí que los alias de tabla acortan mucho las consultas largas sin cambiar el resultado.

**Aplicación real:** en consultas largas con muchas tablas, los alias evitan errores y facilitan la revisión del código.

## Ejercicio 26 – Alias de columna

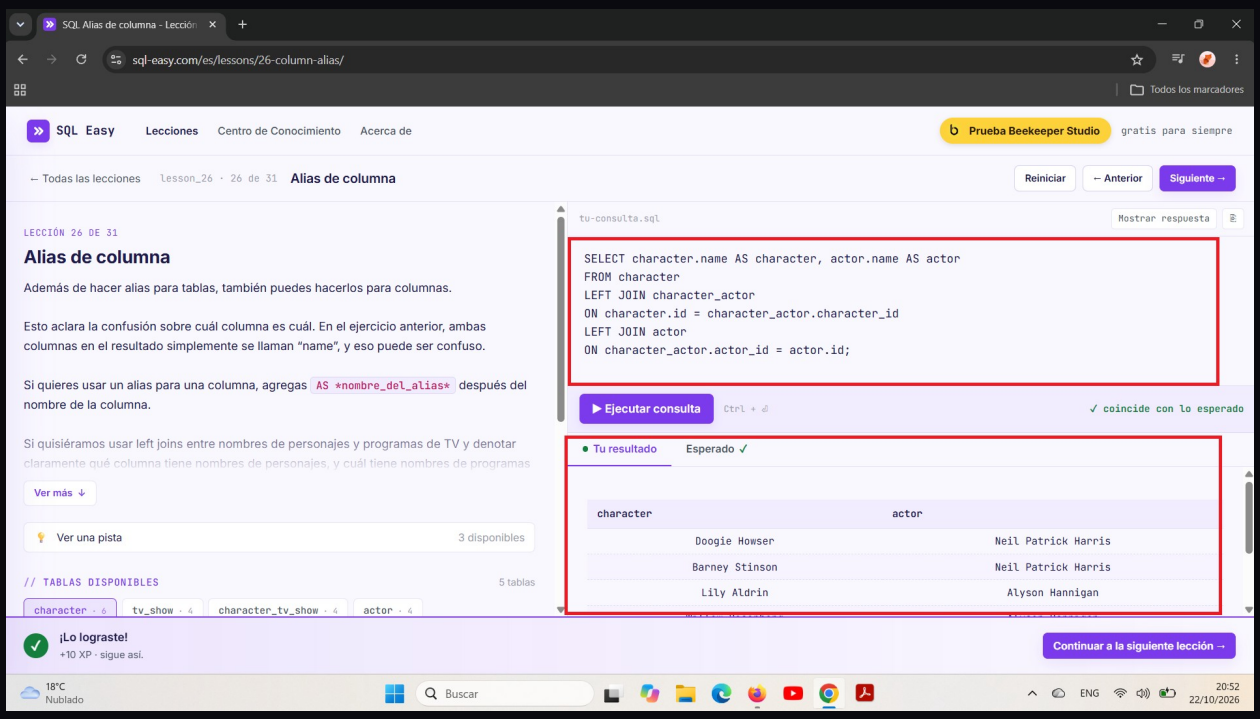
**Objetivo:** mostrar personajes y actores con nombres de columna más claros.

**Concepto SQL:** `AS` también renombra las columnas del resultado.

**Consulta ejecutada:**

```
SELECT character.name AS character, actor.name AS actor
FROM character
LEFT JOIN character_actor
ON character.id = character_actor.character_id
LEFT JOIN actor
ON character_actor.actor_id = actor.id;
```

**Resultado:**




Devuelve los mismos datos del ejercicio 24, con las columnas llamadas character y actor.

**Interpretación:** como las dos columnas se llamaban name, el alias evita la confusión.

**Aprendizaje:** comprendí que AS también renombra columnas y evita confusiones cuando dos se llaman igual.

**Aplicación real:** reportes legibles para otras personas. En inyecciones SQL, los alias junto con UNION se usan para hacer coincidir columnas al extraer datos.

## Ejercicio 27 – Self join

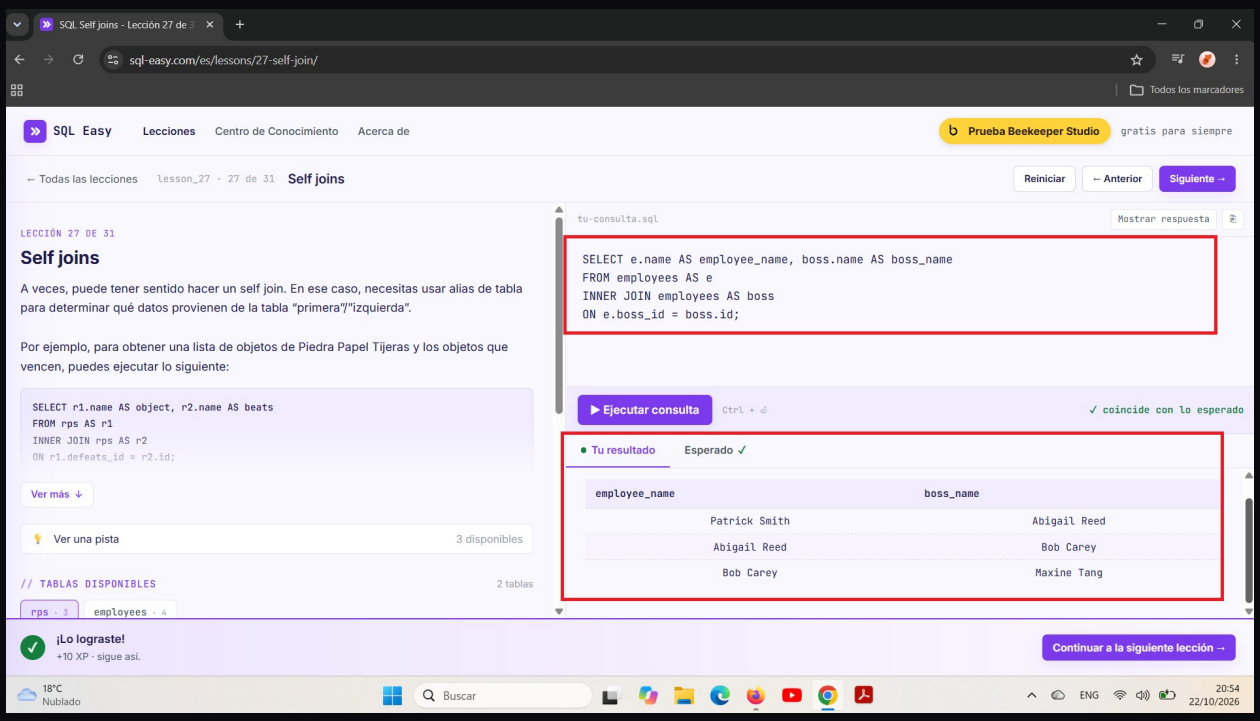
**Objetivo:** mostrar cada empleado junto al nombre de su jefe.

**Concepto SQL:** un self join une una tabla consigo misma usando dos alias distintos (e y boss).

**Consulta ejecutada:**

```
 SELECT e.name AS employee_name, boss.name AS boss_name
FROM employees AS e
INNER JOIN employees AS boss
ON e.boss_id = boss.id;
```

**Resultado:**




Devuelve pares empleado–jefe.

**Interpretación:** la columna boss_id apunta a otro registro de la misma tabla, y el self join permite leer esa relación.

**Aprendizaje:** aprendí que una tabla se puede unir consigo misma usando dos alias distintos, como si fueran dos tablas.

**Aplicación real:** jerarquías como organigramas, cadenas de aprobación o estructuras de áreas guardadas en una misma tabla.

## Ejercicio 28 – LIKE: búsqueda por patrones

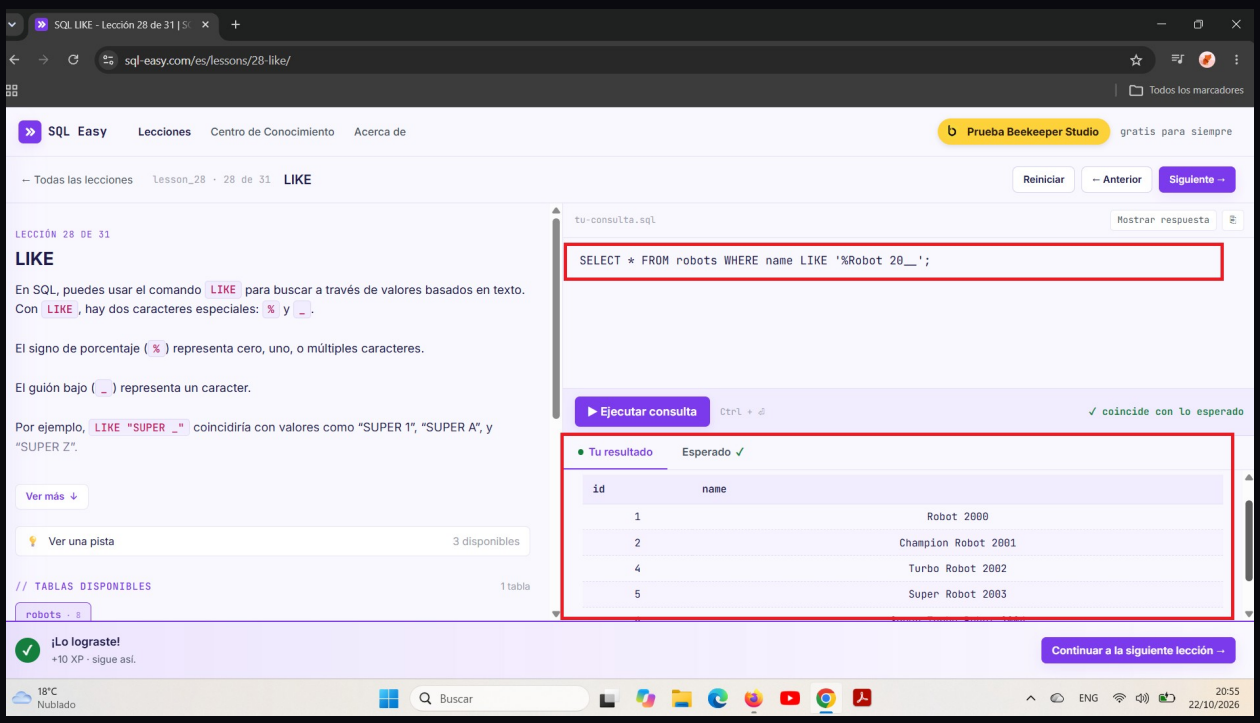
**Objetivo:** encontrar los robots cuyo nombre termina en 'Robot 20' seguido de dos caracteres.

**Concepto SQL:** `LIKE` busca patrones: `%` reemplaza cualquier cantidad de caracteres y `_` exactamente uno.

**Consulta ejecutada:**

```
SELECT * FROM robots WHERE name LIKE '%Robot 20__';
```

**Resultado:**




Devuelve los robots cuyo nombre coincide con el patrón (los de la serie 20XX).

**Interpretación:** `__` exige exactamente dos caracteres después de 'Robot 20'.

**Aprendizaje:** comprendí la diferencia entre los comodines de LIKE: `%` para cualquier cantidad de caracteres y `_` para exactamente uno.

**Aplicación real:** buscadores y filtros por texto parcial. En seguridad sirve para rastrear patrones en logs (URLs que contienen 'admin') y también es un vector de inyección si la entrada no se valida.

## Ejercicio 29 – CASE: lógica condicional

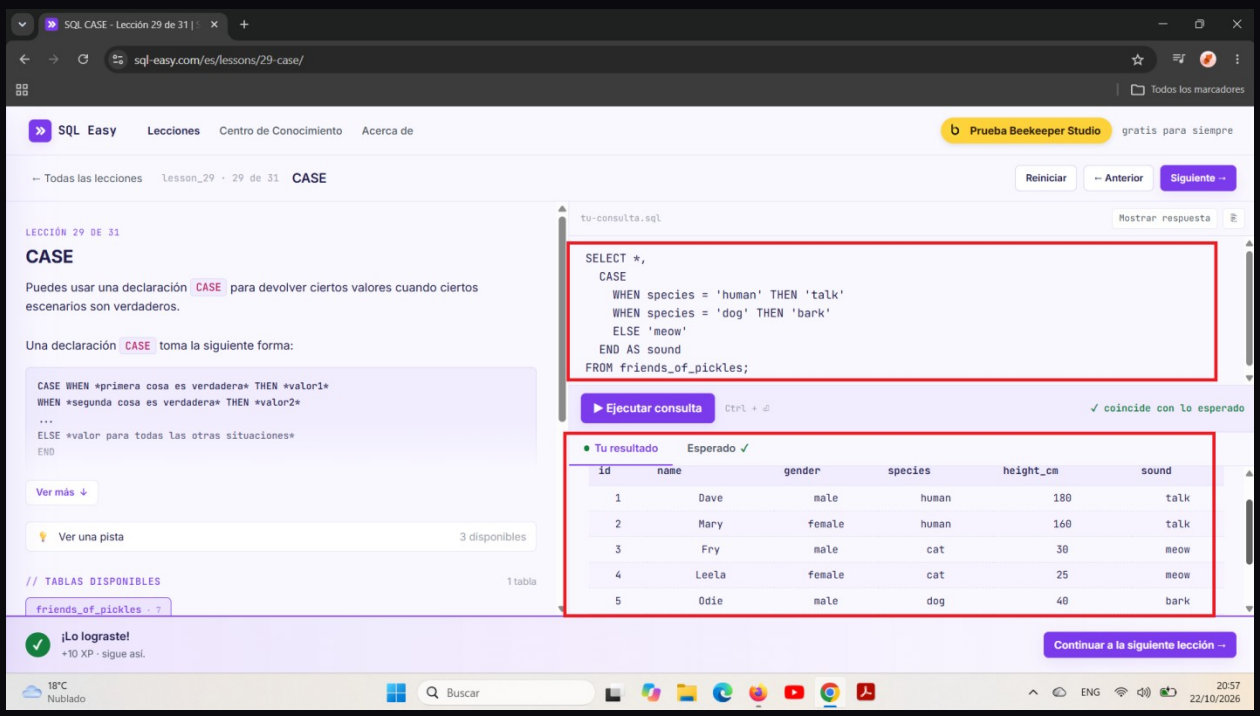
**Objetivo:** agregar una columna que indique el sonido de cada animal según su especie.

**Concepto SQL:** `CASE WHEN ... THEN ... ELSE ... END` genera valores según condiciones, como un if/else.

**Consulta ejecutada:**

```
SELECT *,
  CASE
    WHEN species = 'human' THEN 'talk'
    WHEN species = 'dog' THEN 'bark'
    ELSE 'meow'
  END AS sound
FROM friends_of_pickles;
```

**Resultado:**




Devuelve todas las filas de friends_of_pickles con una columna nueva, sound.

**Interpretación:** la columna se calcula al momento de la consulta; no se guarda en la tabla.

**Aprendizaje:** aprendí que CASE permite aplicar lógica condicional dentro de una consulta y crear columnas nuevas en el resultado.

**Aplicación real:** clasificar datos al vuelo, como asignar niveles de riesgo (alto, medio, bajo) según un puntaje o etiquetar eventos según su tipo.

## Ejercicio 30 – SUBSTR: extraer parte de un texto

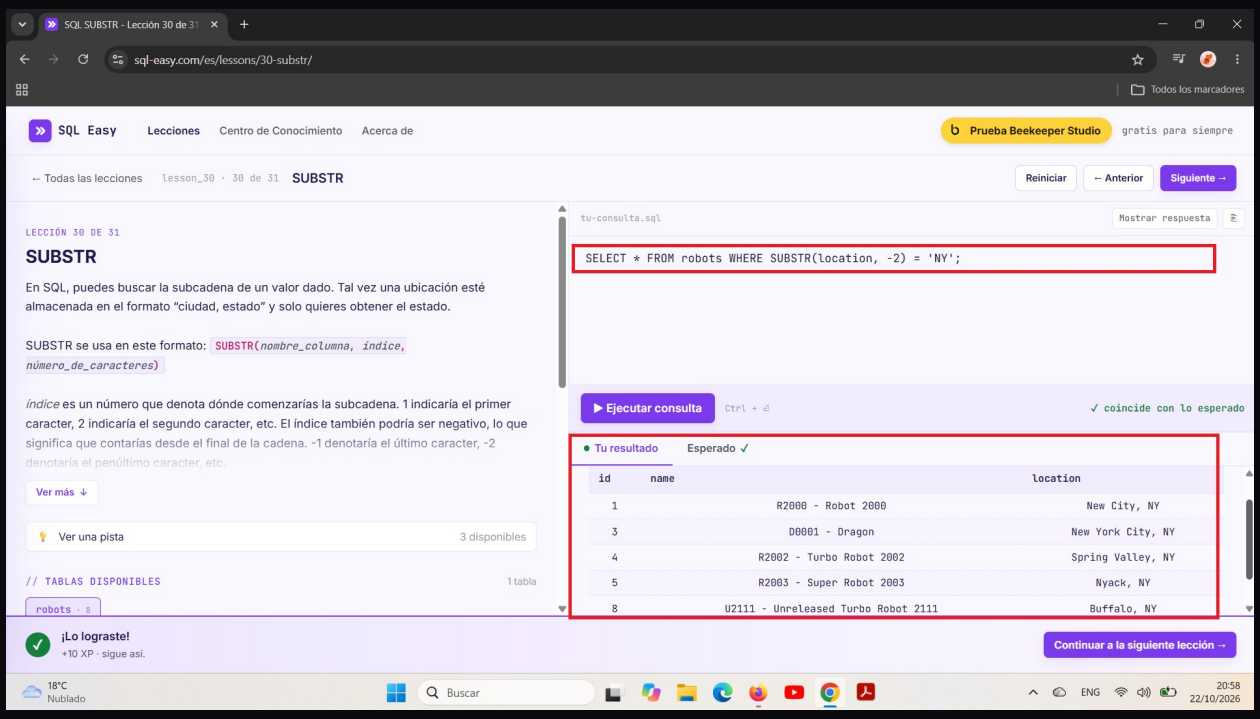
**Objetivo:** encontrar los robots ubicados en el estado de Nueva York.

**Concepto SQL:** `SUBSTR(texto, inicio)` extrae parte de un texto; con -2 toma los dos últimos caracteres.

**Consulta ejecutada:**

```
SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';
```

**Resultado:**




Devuelve los robots cuya location termina en 'NY'.

**Interpretación:** toma los dos últimos caracteres de la ubicación (la sigla del estado) y los compara con 'NY'.

**Aprendizaje:** aprendí a usar SUBSTR con posiciones negativas para tomar caracteres desde el final del texto.

**Aplicación real:** procesar datos con formato fijo, como códigos de área, dominios de email o extensiones de archivo. En inyecciones SQL a ciegas se usa para extraer información carácter por carácter.

## Ejercicio 31 – COALESCE: primer valor no nulo

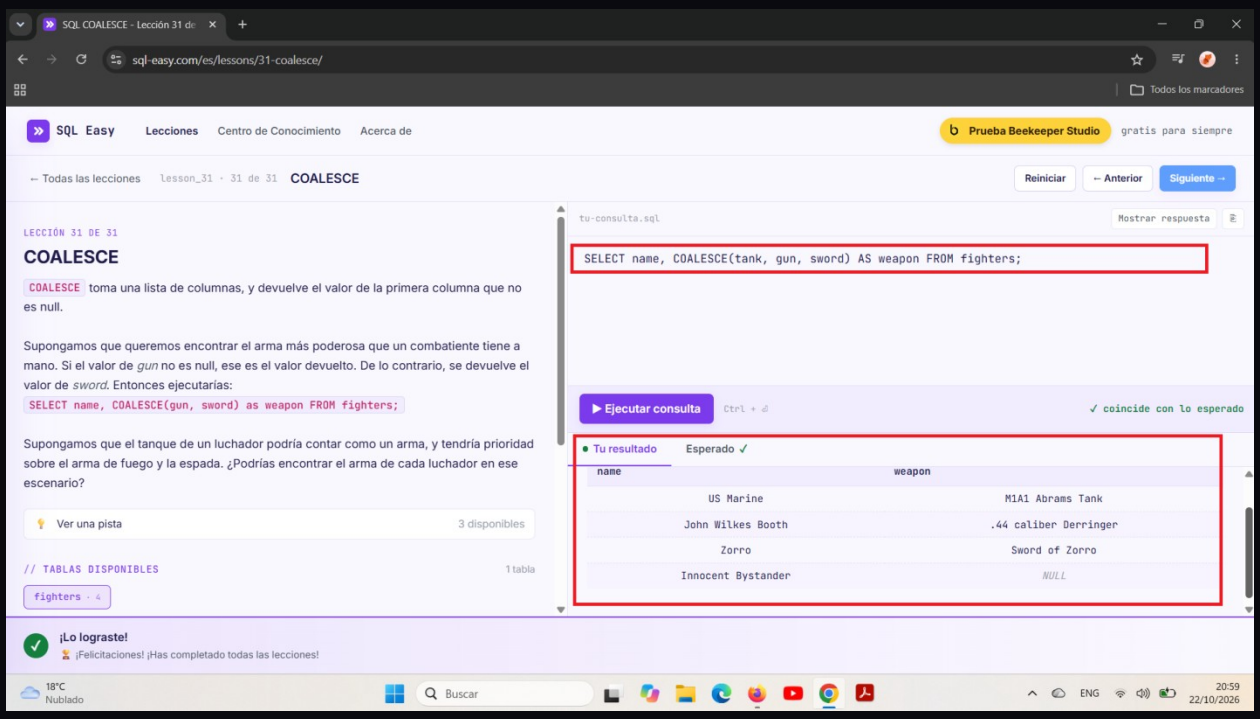
**Objetivo:** mostrar el arma de cada luchador, tomando la primera que tenga cargada.

**Concepto SQL:** `COALESCE()` devuelve el primer valor no NULL de la lista.

**Consulta ejecutada:**

```
SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;
```

**Resultado:**




Devuelve cada luchador con una columna weapon.

**Interpretación:** si tank es NULL pasa a gun, y si también lo es, a sword.

**Aprendizaje:** comprendí que COALESCE recorre la lista de valores y devuelve el primero que no es NULL.

**Aplicación real:** completar datos faltantes con valores alternativos, como mostrar el teléfono principal o, si no hay, el secundario.

## Reflexión y conclusión

### Temas más fáciles

Los ejercicios iniciales (SELECT, WHERE, AND, OR, ORDER BY y LIMIT) me resultaron más simples porque siguen una lógica parecida al lenguaje natural: elegir qué mostrar, de dónde y con qué condición. Las funciones de agregación (COUNT, SUM, AVG y MAX) también fueron directas una vez que entendí que resumen la tabla en un solo valor.

### Temas más retadores

Los más desafiantes fueron los JOIN, sobre todo los múltiples con tabla intermedia y la diferencia entre INNER y LEFT JOIN, y la subconsulta del ejercicio 18. Para resolverlos me ayudó comparar los resultados de ejercicios parecidos y ver qué filas aparecían o desaparecían en cada caso.

### Aplicación en un contexto real

Estas consultas son la base del trabajo con datos en cualquier organización. Desde la ciberseguridad, sirven para analizar logs (contar intentos de login fallidos por usuario con COUNT y GROUP BY), auditar accesos (encontrar usuarios sin rol asignado con LEFT JOIN) y detectar anomalías (montos por encima

del promedio con subconsultas). Además, entender cómo se construye una consulta permite reconocer cómo un atacante manipula cláusulas como WHERE u OR en una inyección SQL, y cómo prevenirlo con consultas parametrizadas y validación de entradas.

## Conclusión general

La actividad me permitió recorrer SQL de forma progresiva, desde consultas simples hasta combinaciones de tablas y funciones. Documentar cada ejercicio me obligó a explicar no solo qué hace cada consulta, sino por qué funciona y para qué sirve en un contexto real. Esta base es la que voy a usar en la segunda parte del reto, al analizar con OWASP ZAP vulnerabilidades como la inyección SQL desde el lado de la aplicación web.

# OWASP ZAP – Primera evaluación de seguridad web

Gin & Juice Shop · Oct 25, 2026 · @Yesica Perez

## Datos de la actividad

| Campo | Detalle |
|---|---|
| Curso | Hacker Women Council – Ethical Hacking |
| Guía | 02 – SQL & Web Security Challenge |
| Parte | OWASP ZAP: primera evaluación de seguridad web |
| Sitio evaluado | https://ginandjuice.shop (sitio de práctica de PortSwigger) |
| Herramienta | OWASP ZAP |
| Alumna | Yesica Pérez |

**Objetivo:** explorar Gin & Juice Shop con OWASP ZAP, identificar alertas de seguridad y documentar un hallazgo principal.

**Qué documentar:** 4 elementos del entorno, 5 de exploración, 3 alertas, 1 ficha técnica de 8 campos y 3 conclusiones.

- Usá los hallazgos que te dé tu propio escaneo; los de la guía son solo un ejemplo.
- Reemplazá cada texto en itálica por tu captura o tu respuesta.

## 1. Prepara tu entorno

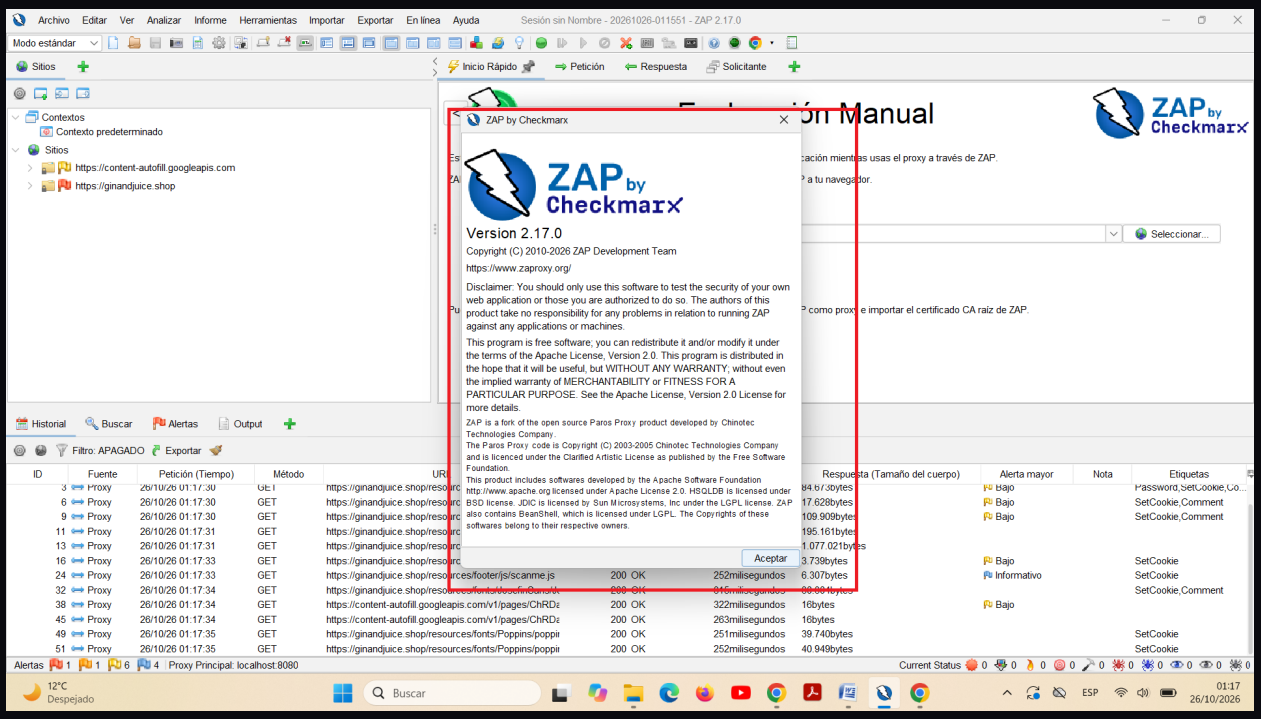
Documentá 4 elementos.

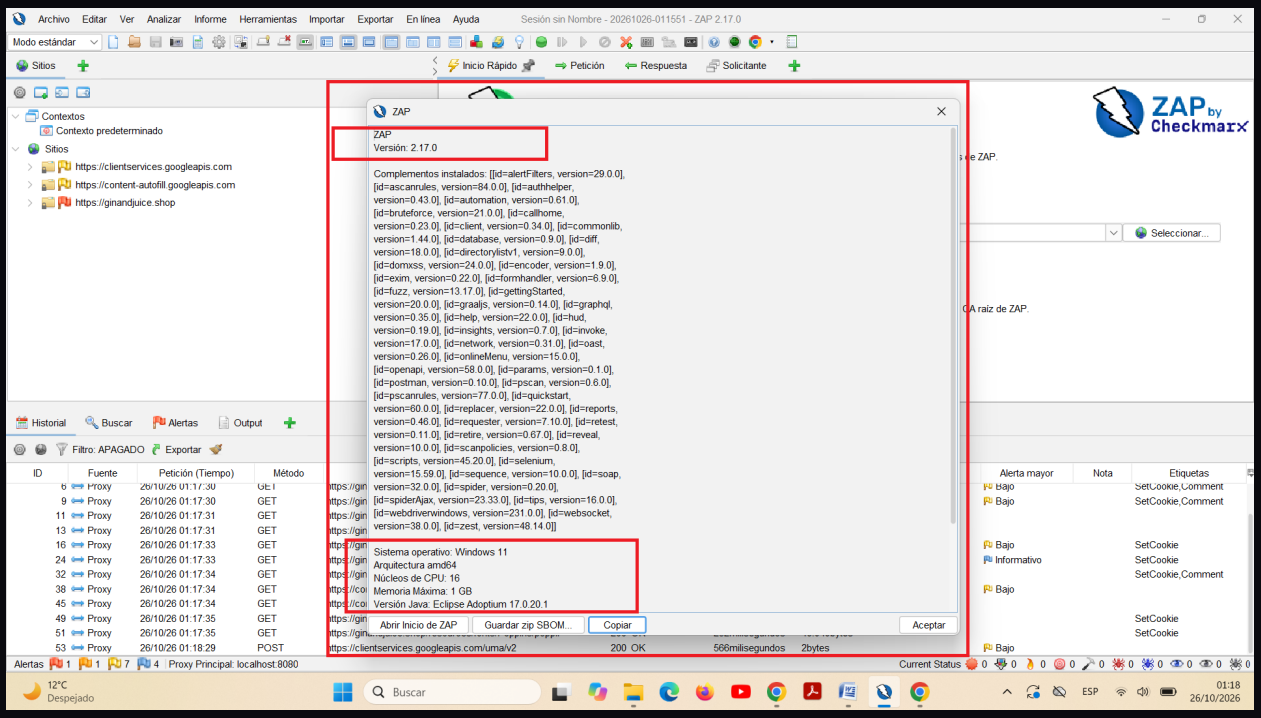
### 1.1 ZAP abierto

*Interfaz principal de ZAP*

**Descripción:** *La versión de ZAP 2.17.0  Sistema operativo Windows 11*

**Evidencias:**

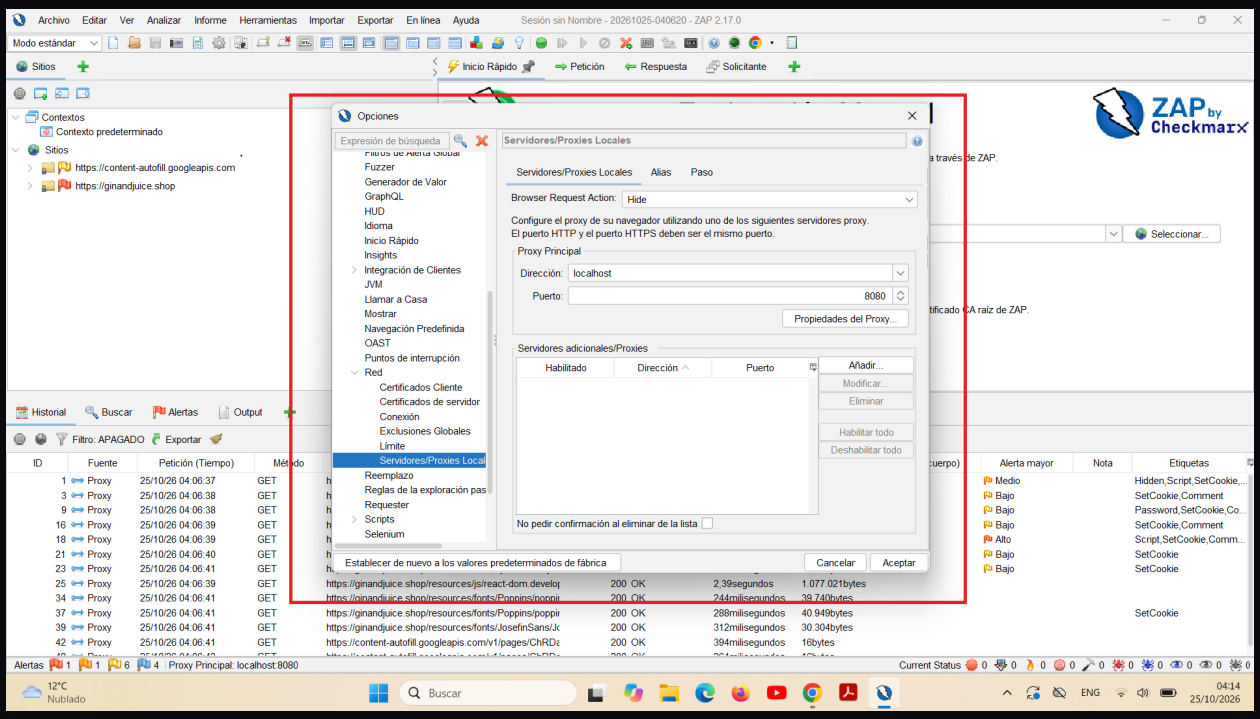



## 1.2 Navegador configurado

**Descripción:** Utilicé el navegador Chrome junto con la opción Manual Explore de ZAP, que abre el navegador ya configurado para enviar todo el tráfico a través del proxy local 127.0.0.1:8080. Además, tuve que instalar Eclipse Adoptium (Java) para que ZAP funcionara correctamente y sin inconvenientes.

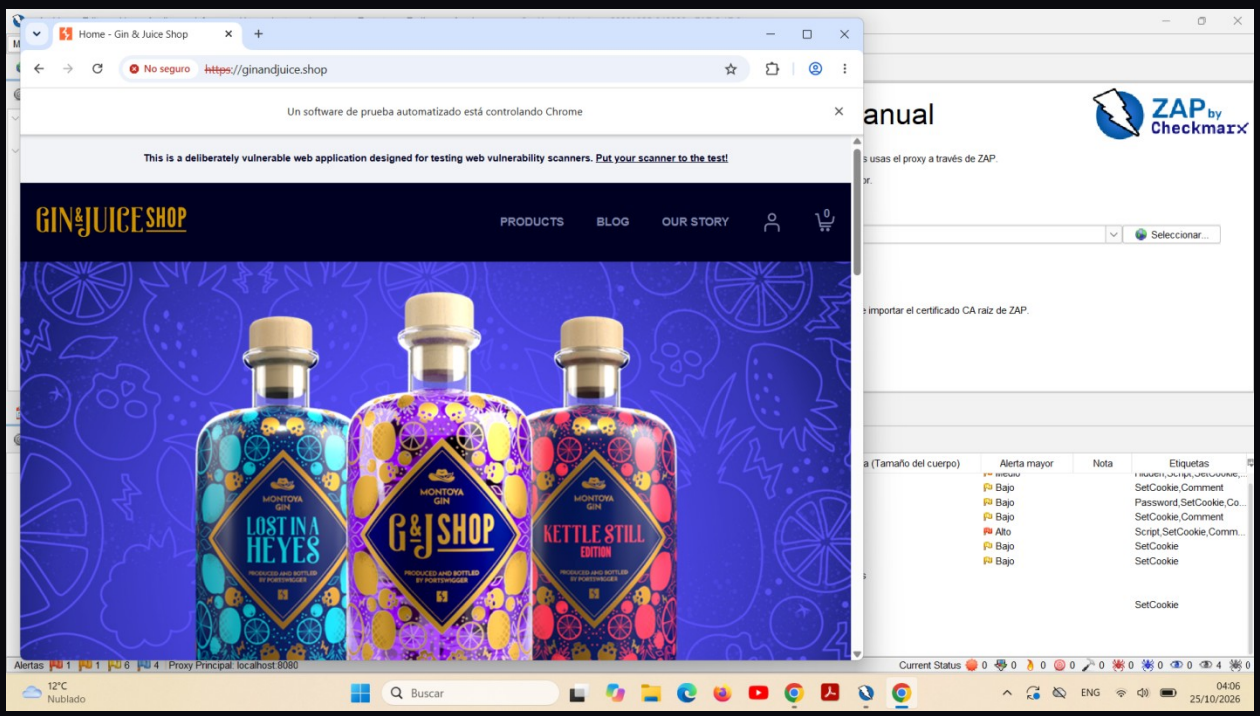
**Evidencias:**




## 1.3 Acceso a https://ginandjuice.shop/

Sitio

**Evidencia:**

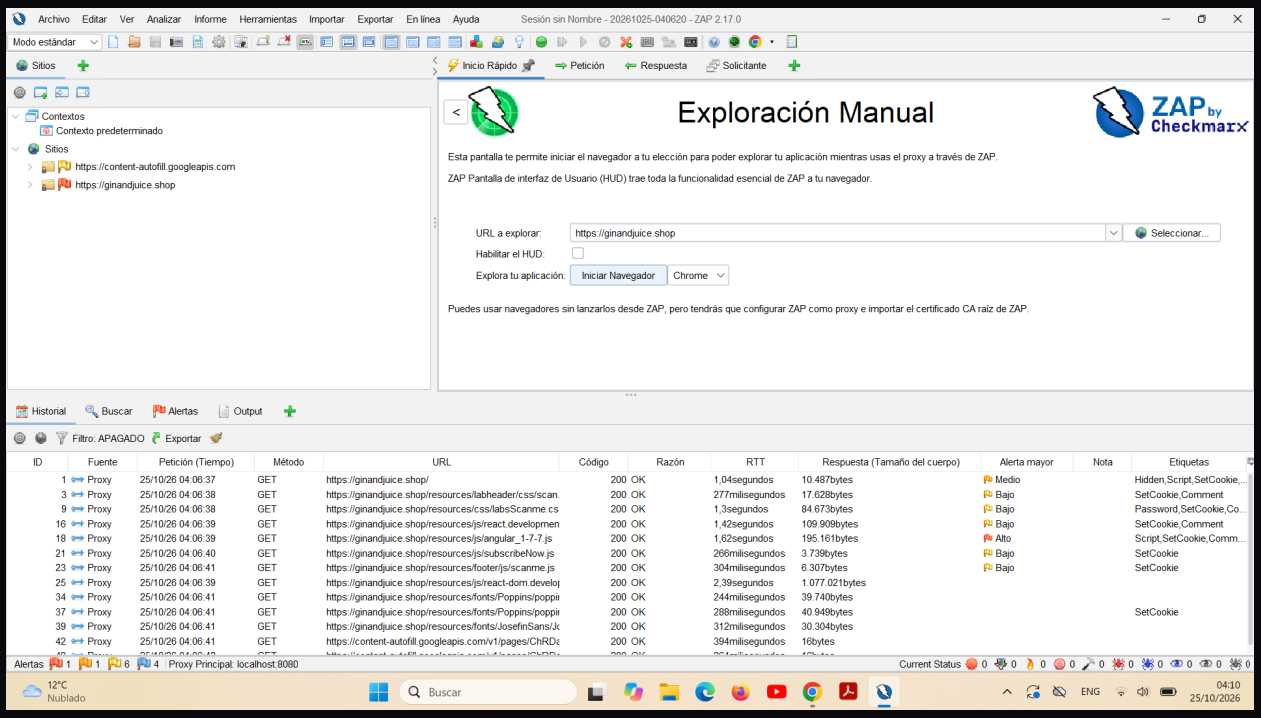


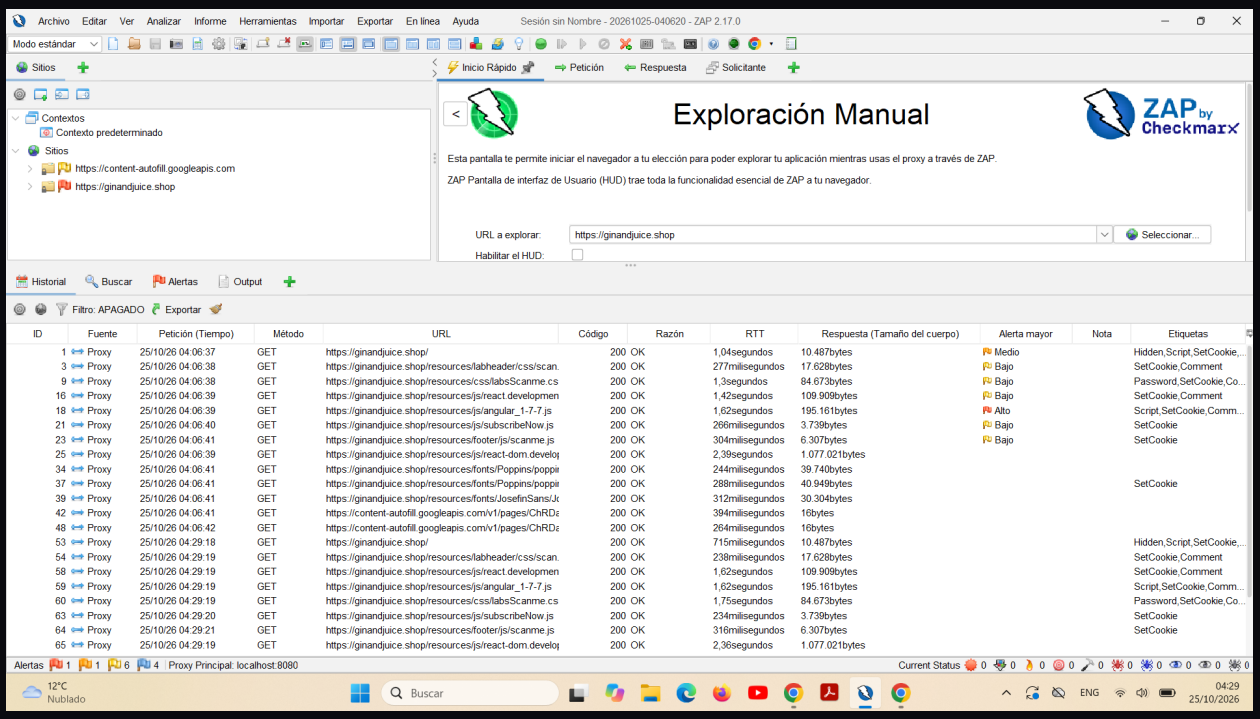
## 1.4 Verificación del tráfico en ZAP

Pestaña *History con las peticiones GET al sitio.*

**Descripción:** La captura muestra las peticiones registradas en la pestaña Historial de ZAP. Se ven varias peticiones con el método GET hacia distintas URLs de ginandjuice.shop, cada una con su código de respuesta 200 OK, lo que confirma que el tráfico del navegador pasa a través del proxy de ZAP.

**Evidencias:**



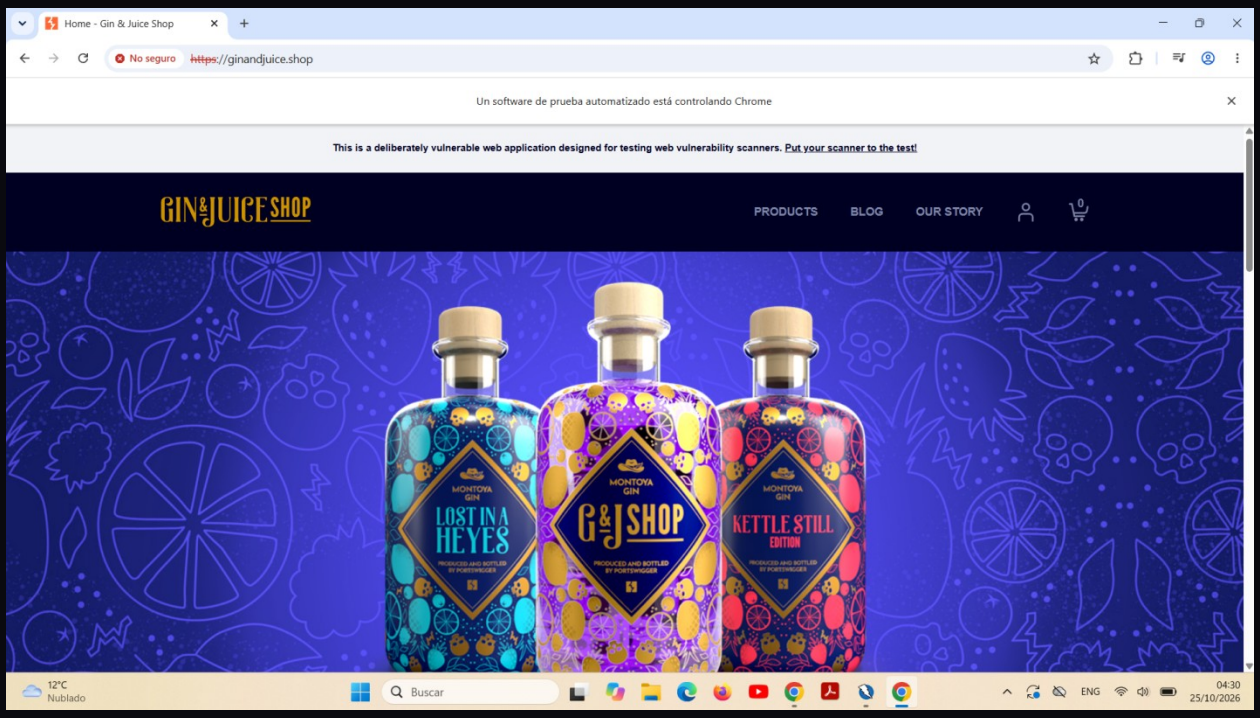



## 2. Explora la aplicación

Documentación de 5 elementos: la página principal, su tráfico en ZAP y 3 secciones más del sitio.

## 2.1 Navegación en el sitio

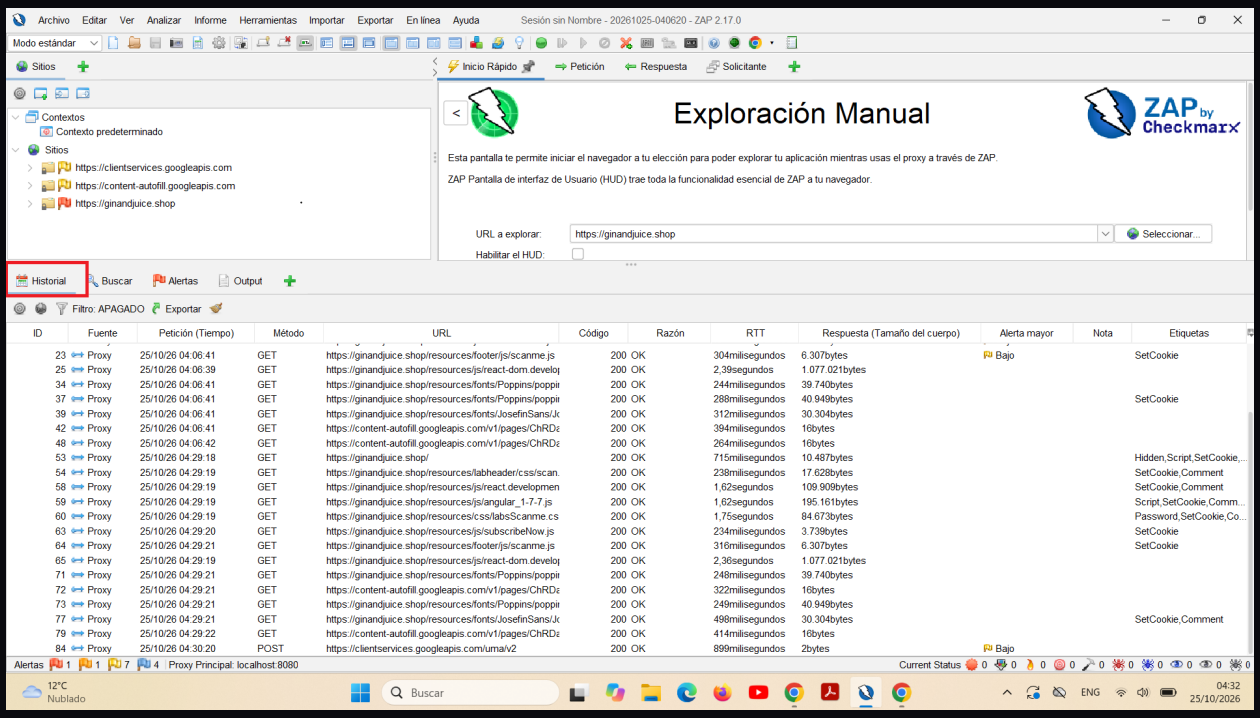
**Evidencias:**



## 2.2 Tráfico HTTP en ZAP

**Descripción:** La captura muestra el tráfico HTTP registrado en la pestaña Historial de ZAP mientras se navega por el sitio. Se observan varias peticiones con el método GET hacia distintas URLs de ginandjuice.shop (páginas, hojas de estilo, imágenes y scripts), cada una con su código de respuesta 200 OK. Esto confirma que todo el tráfico del navegador pasa a través del proxy de ZAP y queda registrado para su análisis.

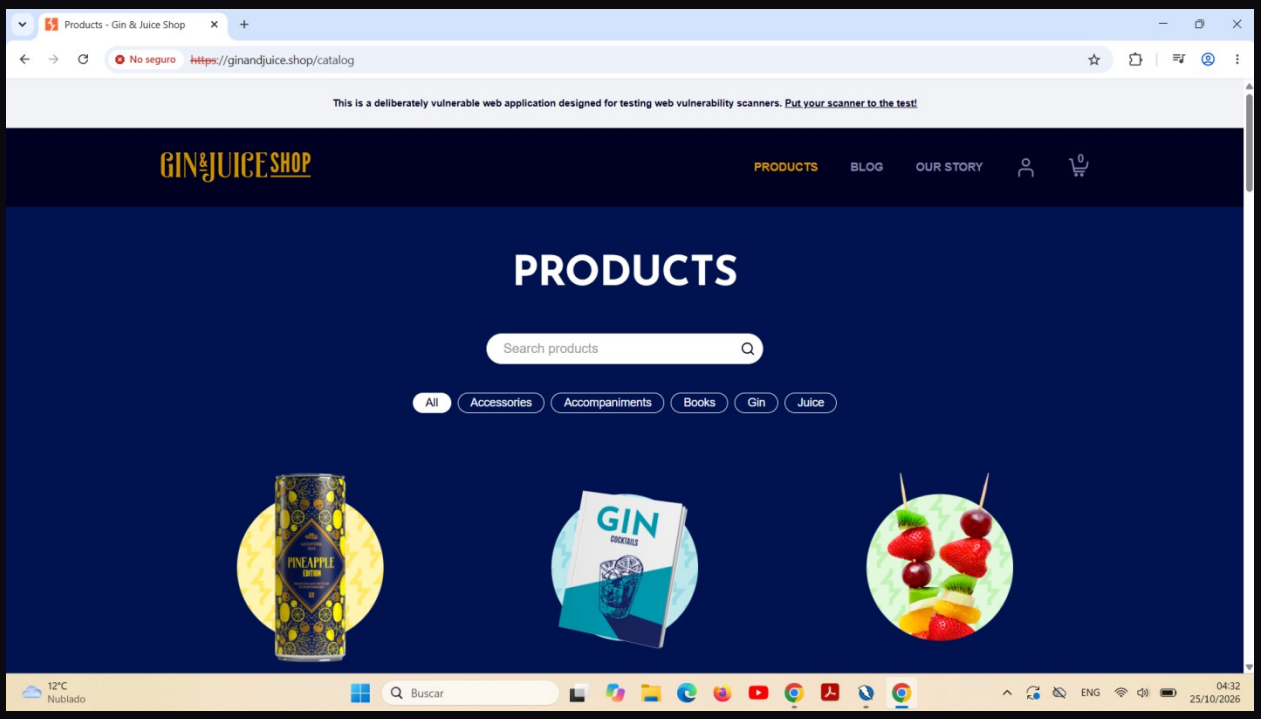
**Evidencia:**

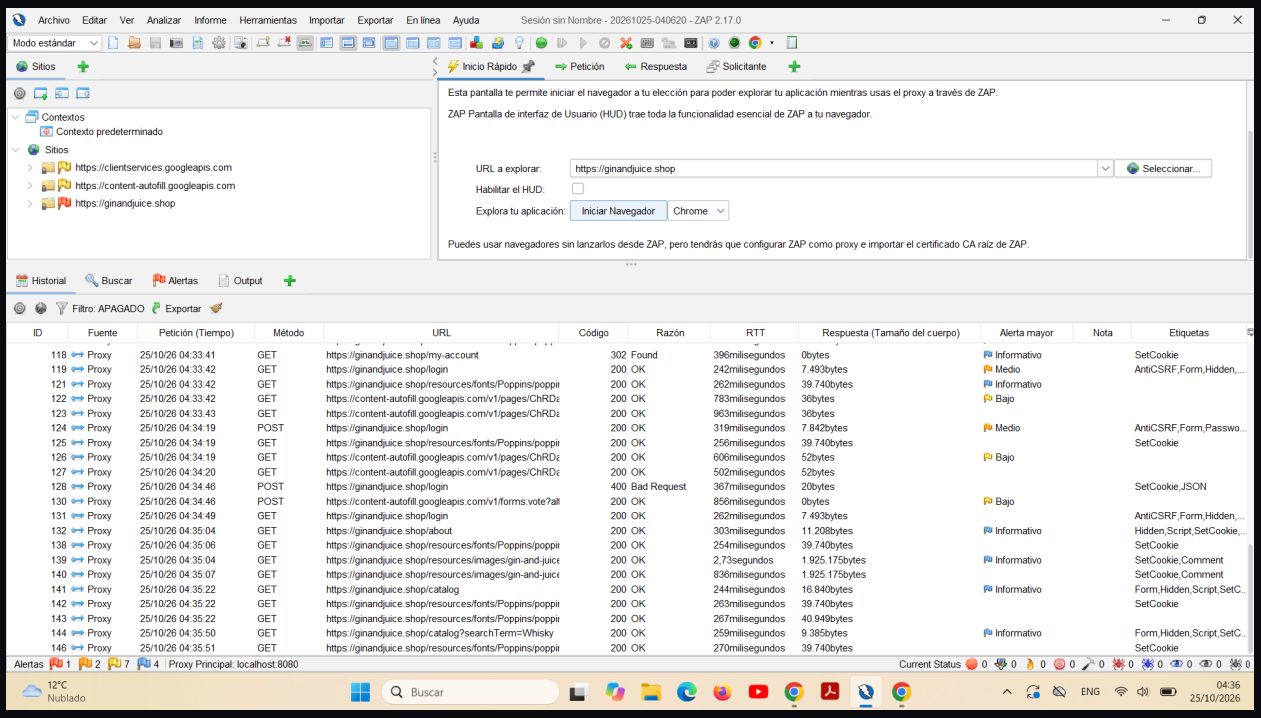



## 2.3 Sección explorada: *(ej.: página de inicio de sesión)*

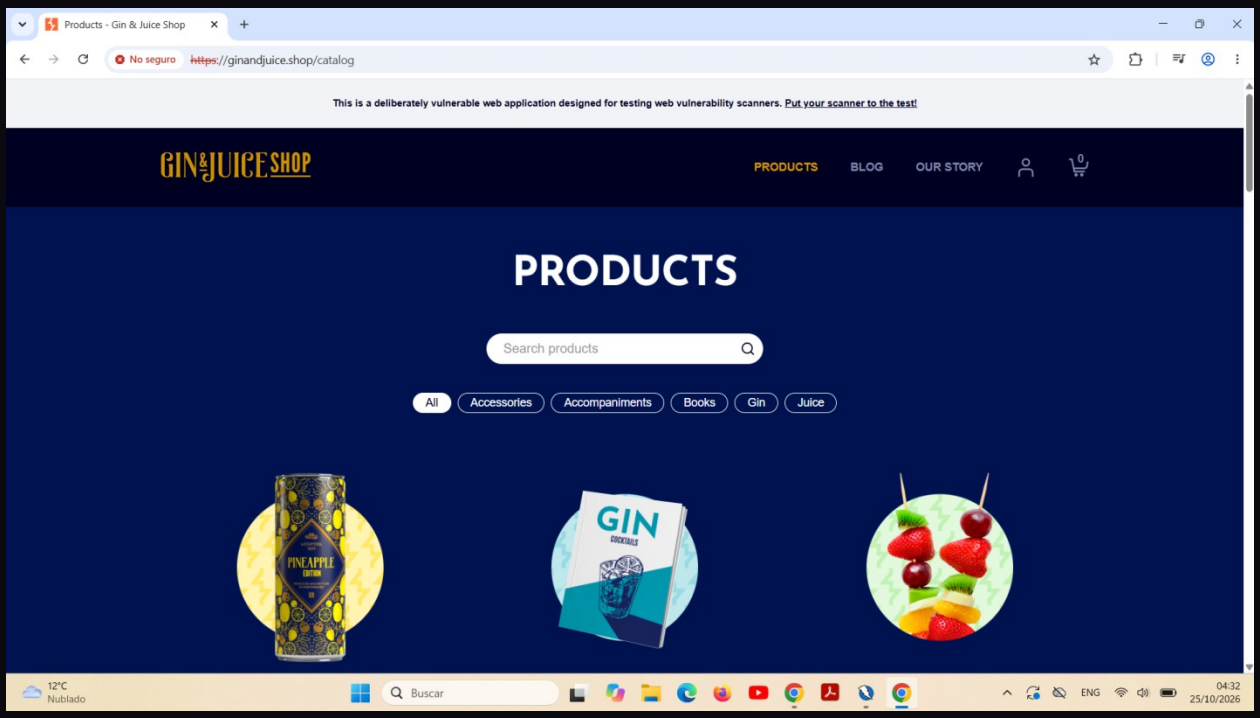
**Qué observé:** Al iniciar sesión, además de las peticiones GET que cargan la página de login, aparece una petición con el método **POST** hacia la URL `https://ginandjuice.shop/login` (IDs 124 y 128). El POST es el que envía los datos del formulario (usuario y contraseña) al servidor, a diferencia del GET que solo muestra la página. También se observan las etiquetas AntiCSRF, Form y Password, que indican que ZAP detectó un formulario de inicio de sesión.

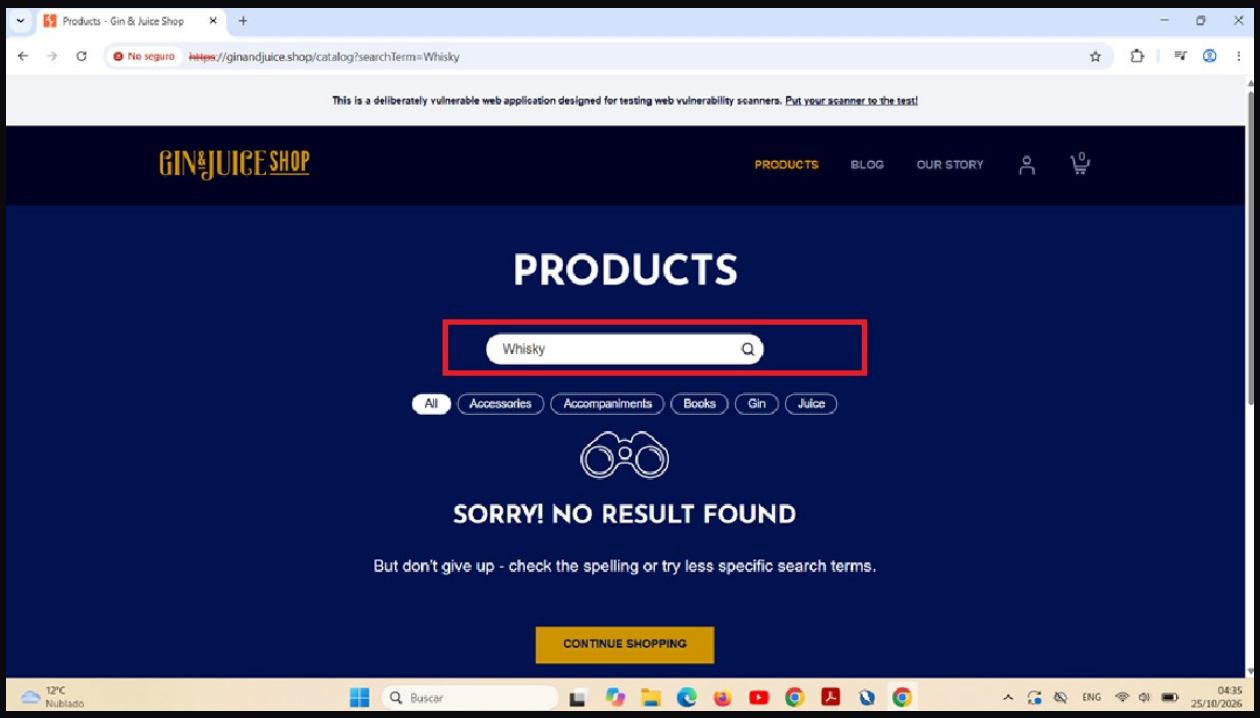
**Evidencias:**

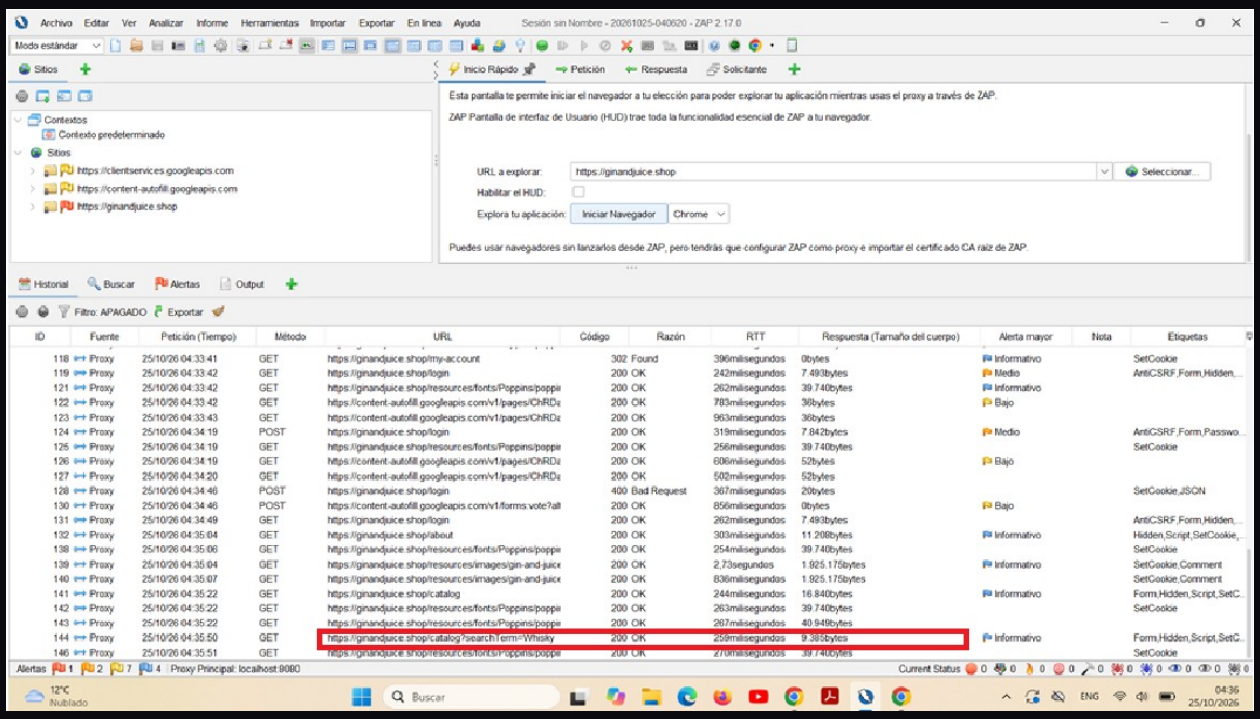

Products - Gin & Juice Shop
No seguro
https://ginandjuice.shop/catalog
This is a deliberately vulnerable web application designed for testing web vulnerability scanners. Put your scanner to the test!
GIN & JUICE SHOP
PRODUCTS
BLOG
OUR STORY
PRODUCTS
Search products
All
Accessories
Accompaniments
Books
Gin
Juice
PINEAPPLE
GIN
12°C
Nublado
Buscar
ENG
04:32
25/10/2026


Archivo Editar Ver Analizar Informe Herramientas Importar Exportar En línea Ayuda
Sesión sin Nombre - 20261025-040620 - ZAP 2.17.0
Modo estándar
Sitios
Inicio Rápido
Petición
Respuesta
Solicitante
Esta pantalla te permite iniciar el navegador a tu elección para poder explorar tu aplicación mientras usas el proxy a través de ZAP.
ZAP Pantalla de interfaz de Usuario (HUD) trae toda la funcionalidad esencial de ZAP a tu navegador.
Contextos
Contexto predeterminado
Sitios
https://clientservices.googleapis.com
https://content-autofill.googleapis.com
https://ginandjuice.shop
URL a explorar: https://ginandjuice.shop
Seleccionar...
Habilitar el HUD:
Explora tu aplicación: Iniciar Navegador Chrome
Puedes usar navegadores sin lanzarlos desde ZAP, pero tendrás que configurar ZAP como proxy e importar el certificado CA raíz de ZAP.
Historial
Buscar
Alertas
Output
Filtro: APAGADO
Exportar
ID Fuente Petición (Tiempo) Método URL Código Razón RTT Respuesta (Tamaño del cuerpo) Alerta mayor Nota Etiquetas
118 Proxy 25/10/26 04:33:41 GET https://ginandjuice.shop/my-account 302 Found 396milisegundos 0bytes Informativo SetCookie
119 Proxy 25/10/26 04:33:42 GET https://ginandjuice.shop/login 200 OK 242milisegundos 7.493bytes Medio AntiCSRF,Form,Hidden,...
121 Proxy 25/10/26 04:33:42 GET https://ginandjuice.shop/resources/fonts/Poppins/poppi 200 OK 262milisegundos 39.740bytes Informativo
122 Proxy 25/10/26 04:33:42 GET https://content-autofill.googleapis.com/v1/pages/ChRDa 200 OK 783milisegundos 36bytes Bajo
123 Proxy 25/10/26 04:33:43 GET https://content-autofill.googleapis.com/v1/pages/ChRDa 200 OK 963milisegundos 36bytes
124 Proxy 25/10/26 04:34:19 POST https://ginandjuice.shop/login 200 OK 319milisegundos 7.842bytes Medio AntiCSRF,Form,Passwo...
125 Proxy 25/10/26 04:34:19 GET https://ginandjuice.shop/resources/fonts/Poppins/poppi 200 OK 256milisegundos 39.740bytes SetCookie
126 Proxy 25/10/26 04:34:19 GET https://content-autofill.googleapis.com/v1/pages/ChRDa 200 OK 606milisegundos 52bytes Bajo
127 Proxy 25/10/26 04:34:20 GET https://content-autofill.googleapis.com/v1/pages/ChRDa 200 OK 502milisegundos 52bytes
128 Proxy 25/10/26 04:34:46 POST https://ginandjuice.shop/login 400 Bad Request 367milisegundos 20bytes SetCookie,JSON
130 Proxy 25/10/26 04:34:46 POST https://content-autofill.googleapis.com/v1/forms:vote?all 200 OK 856milisegundos 0bytes Bajo
131 Proxy 25/10/26 04:34:49 GET https://ginandjuice.shop/login 200 OK 262milisegundos 7.493bytes AntiCSRF,Form,Hidden,...
132 Proxy 25/10/26 04:35:04 GET https://ginandjuice.shop/about 200 OK 303milisegundos 11.208bytes Informativo Hidden,Script,SetCookie,...
138 Proxy 25/10/26 04:35:06 GET https://ginandjuice.shop/resources/fonts/Poppins/poppi 200 OK 254milisegundos 39.740bytes SetCookie
139 Proxy 25/10/26 04:35:04 GET https://ginandjuice.shop/resources/images/gin-and-juic 200 OK 2,73segundos 1.925.175bytes Informativo SetCookie,Comment
140 Proxy 25/10/26 04:35:07 GET https://ginandjuice.shop/resources/images/gin-and-juic 200 OK 836milisegundos 1.925.175bytes SetCookie,Comment
141 Proxy 25/10/26 04:35:22 GET https://ginandjuice.shop/catalog 200 OK 244milisegundos 16.840bytes Informativo Form,Hidden,Script,SetC...
142 Proxy 25/10/26 04:35:22 GET https://ginandjuice.shop/resources/fonts/Poppins/poppi 200 OK 263milisegundos 39.740bytes SetCookie
143 Proxy 25/10/26 04:35:22 GET https://ginandjuice.shop/resources/fonts/Poppins/poppi 200 OK 267milisegundos 40.949bytes
144 Proxy 25/10/26 04:35:50 GET https://ginandjuice.shop/catalog?searchTerm=Whisky 200 OK 259milisegundos 9.385bytes Informativo Form,Hidden,Script,SetC...
146 Proxy 25/10/26 04:35:51 GET https://ginandjuice.shop/resources/fonts/Poppins/poppi 200 OK 270milisegundos 39.740bytes SetCookie
Alertas 1 2 7 4
Proxy Principal: localhost:8080
Current Status
12°C
Nublado
Buscar
ENG
04:36
25/10/2026

## 2.4 Sección explorada: búsqueda y catálogo de productos.



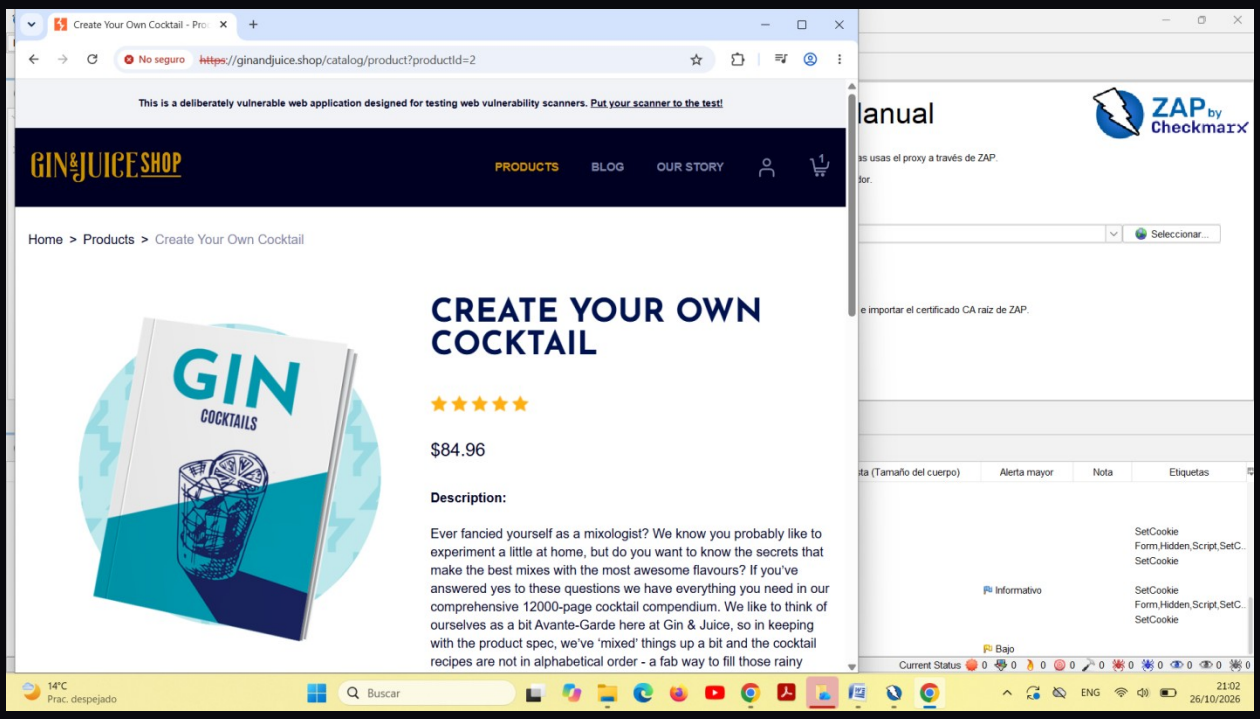



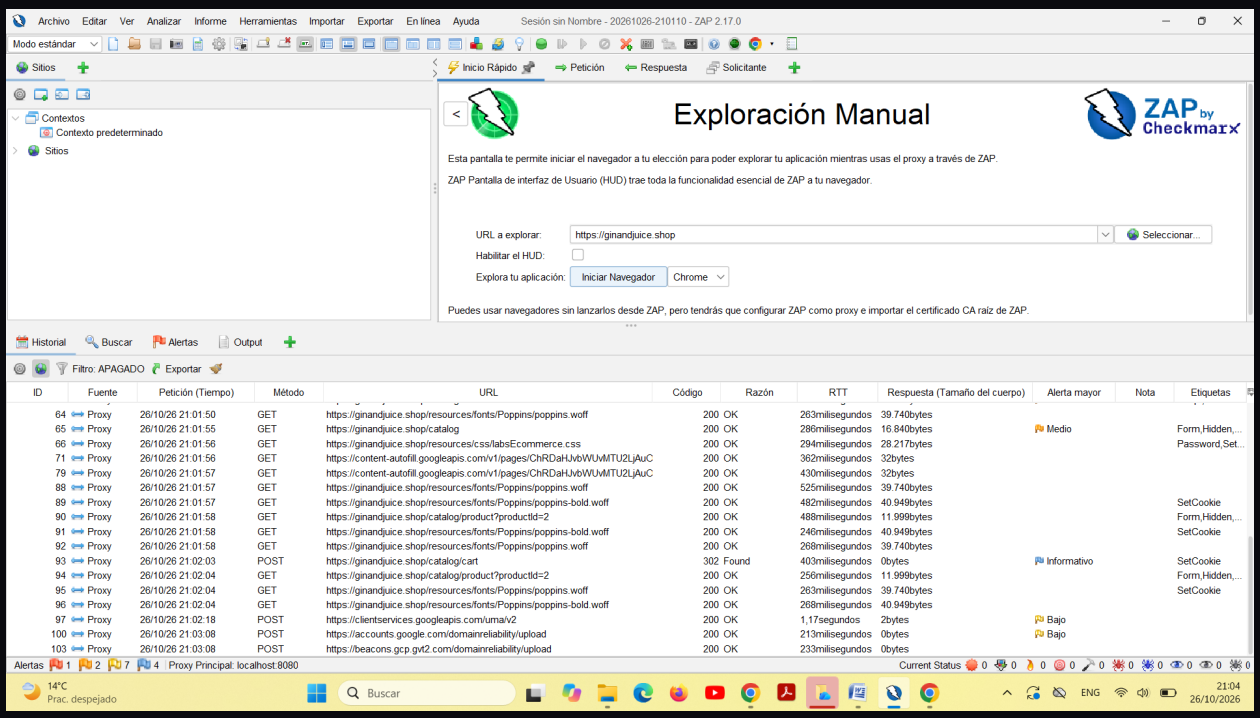



**Qué observé:** Al buscar productos y entrar a un producto, las peticiones viajan con el método GET y llevan parámetros en la URL: `catalog?searchTerm=Whisky` para la búsqueda y `catalog/product?productId=3` para el detalle de un producto. Estos parámetros (searchTerm y productId) son los valores que el usuario puede modificar, y por eso son los puntos donde ZAP después prueba vulnerabilidades como inyección SQL o XSS.

## 2.5 Sección explorada: *carrito de compras.*

**Evidencias:**

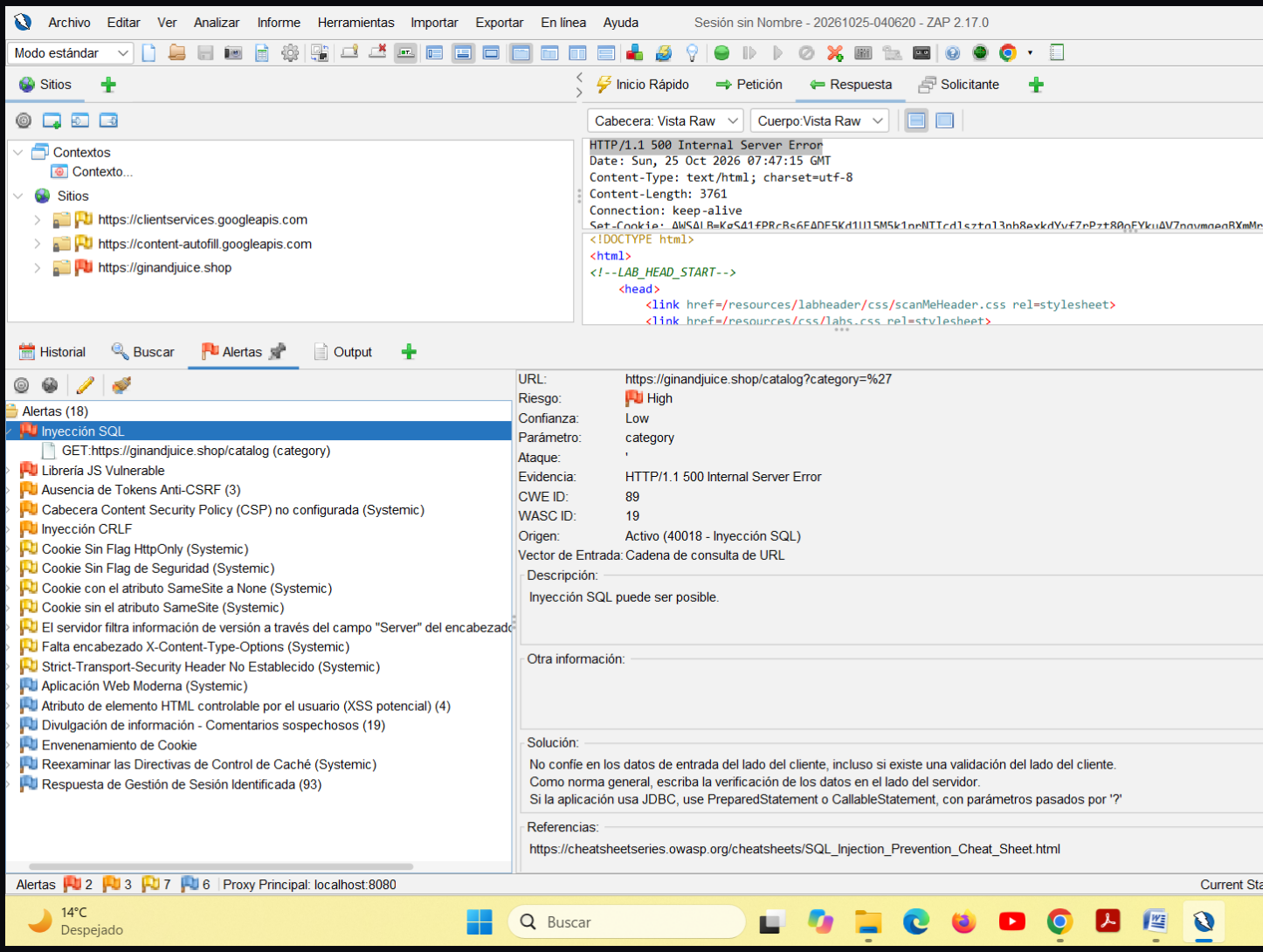



**Qué observé:** Al entrar a un producto y agregarlo al carrito, se generan dos tipos de peticiones. Primero un GET a `catalog/product?productId=2`, que carga el producto con su parámetro productId en la URL. Luego, al agregarlo, aparece una petición con el método **POST** hacia `catalog/add` (ID 93, con respuesta 302 Found), que es la que envía la acción de agregar el producto al servidor. Esto muestra la diferencia entre el GET, que solo pide información, y el POST, que modifica el estado del carrito.

## 3. Analiza las alertas

Documentación de 3 alertas de diferentes tipos y distinto nivel de riesgo (alto, medio, bajo).
**Evidencias:**




| N.º | Alerta | Nivel de riesgo | URL afectada | Qué significa |
|---|---|---|---|---|
| 1 | Inyección SQL | Alto | https://ginandjuice.shop/catalog (parámetro category) | El parámetro permite modificar la consulta SQL que ejecuta el servidor |
| 2 | Cabecera Content Security Policy (CSP) no | Medio | Varias URLs (alerta sistémica) | El sitio no limita desde dónde se pueden cargar scripts y otros recursos |

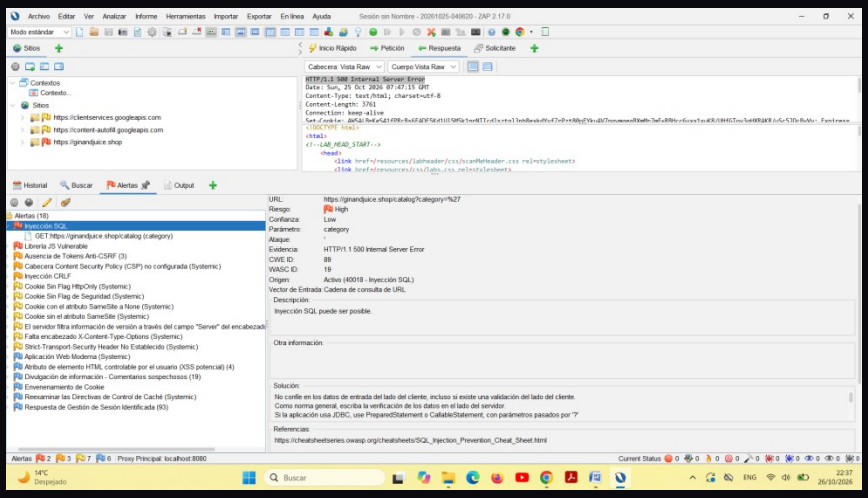
| N.º | Alerta | Nivel de riesgo | URL afectada | Qué significa |
|---|---|---|---|---|
| | configurada | | | |
| 3 | Cookie sin flag HttpOnly | Bajo | Varias URLs (alerta sistémica) | Una cookie puede leerse desde JavaScript en el navegador |

## Alerta 1: Inyección SQL (riesgo alto)

**Explicación:** ZAP detectó que el parámetro category de la página de catálogo no se valida correctamente: al enviar valores manipulados, la respuesta del servidor cambia, lo que indica que el valor se inserta directamente en una consulta SQL. Un atacante podría usarlo para leer o modificar información de la base de datos. Es la vulnerabilidad más grave del escaneo y se desarrolla en la sección 4.

**Evidencia:**

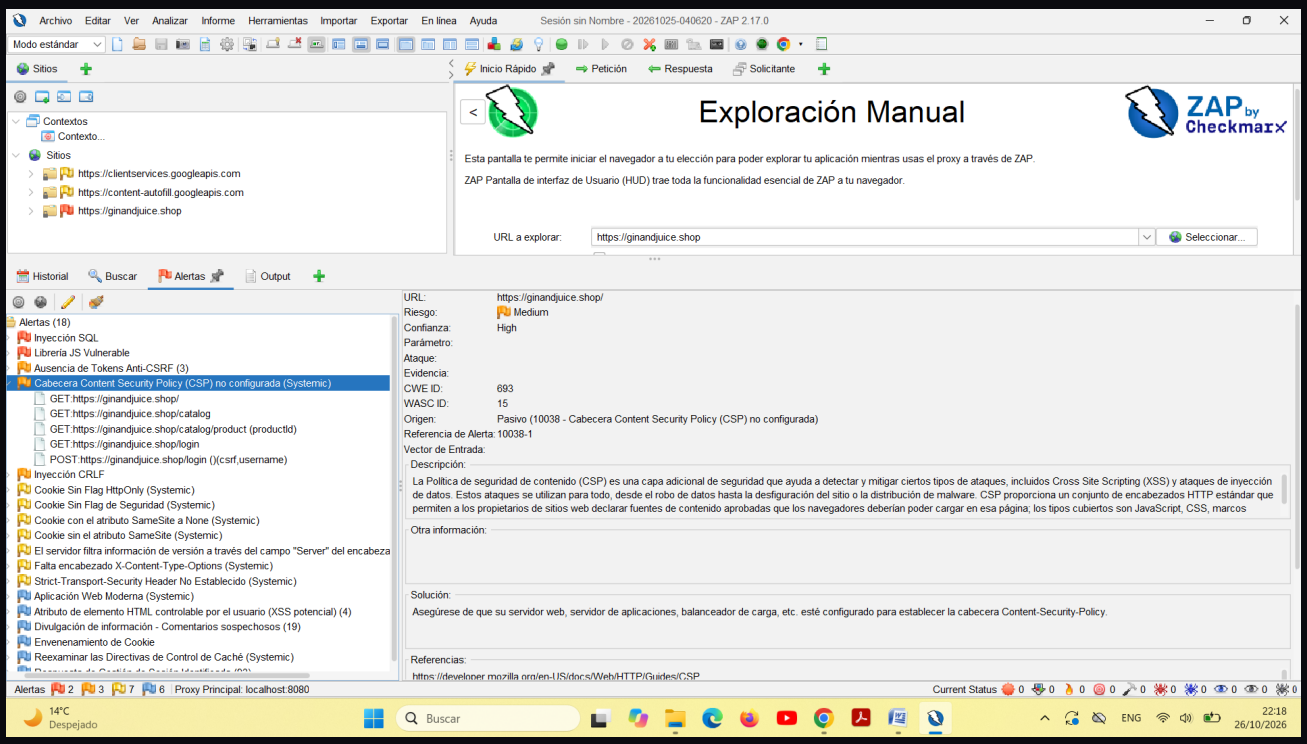
Riesgo: High • Confianza: Low • Parámetro: category • CWE ID: 89 • WASC ID: 19 • Origen: Activo (40018 – Inyección SQL), con su descripción y solución.



## Alerta 2: Cabecera Content Security Policy (CSP) no configurada (riesgo medio)

**Evidencia:**

Riesgo: **Medium** · CWE ID: 693 · WASC ID: 15 · Origen: Pasivo (10038 – Cabecera Content Security Policy no configurada) con la descripción y la solución.

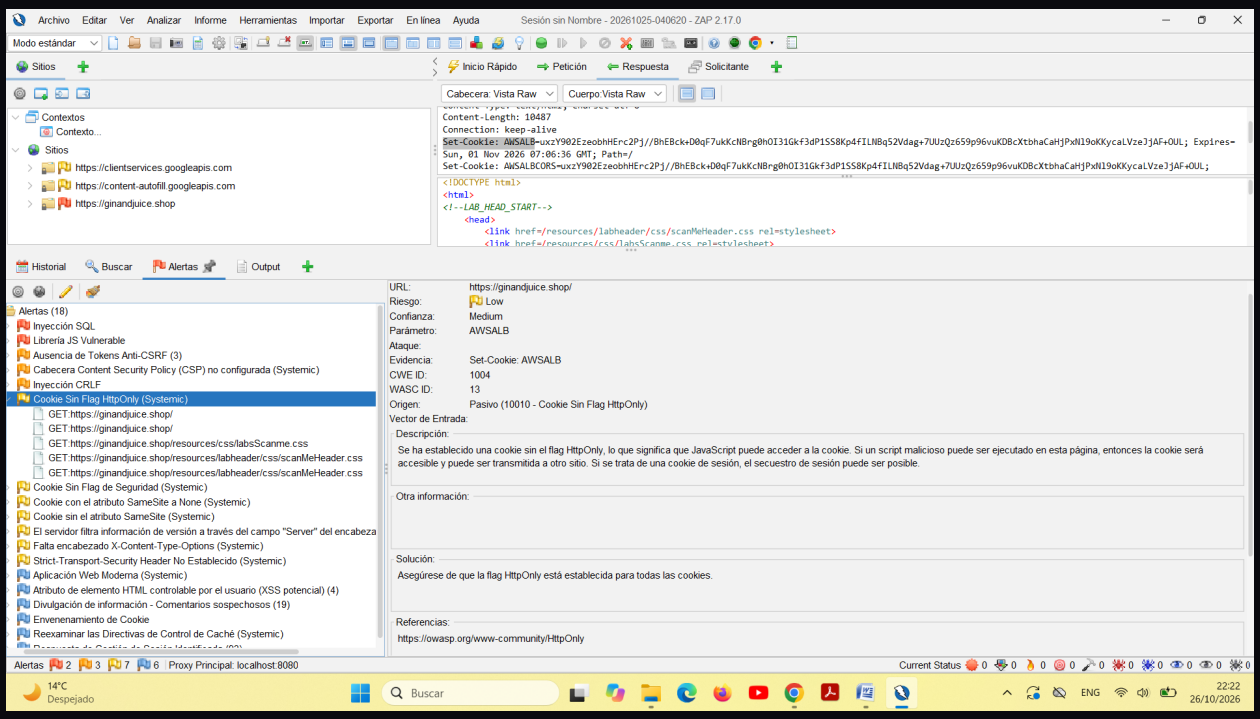



**Explicación:** las respuestas del sitio no incluyen la cabecera Content-Security-Policy, que le indica al navegador de qué orígenes puede cargar scripts, estilos e imágenes. Sin esta política, si existe una vulnerabilidad de Cross-Site Scripting (XSS), el navegador ejecutaría código malicioso sin restricciones. Es una medida de defensa en profundidad que se corrige configurando la cabecera en el servidor.

## Alerta 3: Cookie sin flag HttpOnly (riesgo bajo)

Riesgo: **Low** · Parámetro: AWSALB · CWE ID: 1004 · WASC ID: 13 · Origen: Pasivo (10010 – Cookie Sin Flag HttpOnly), con su descripción y solución.

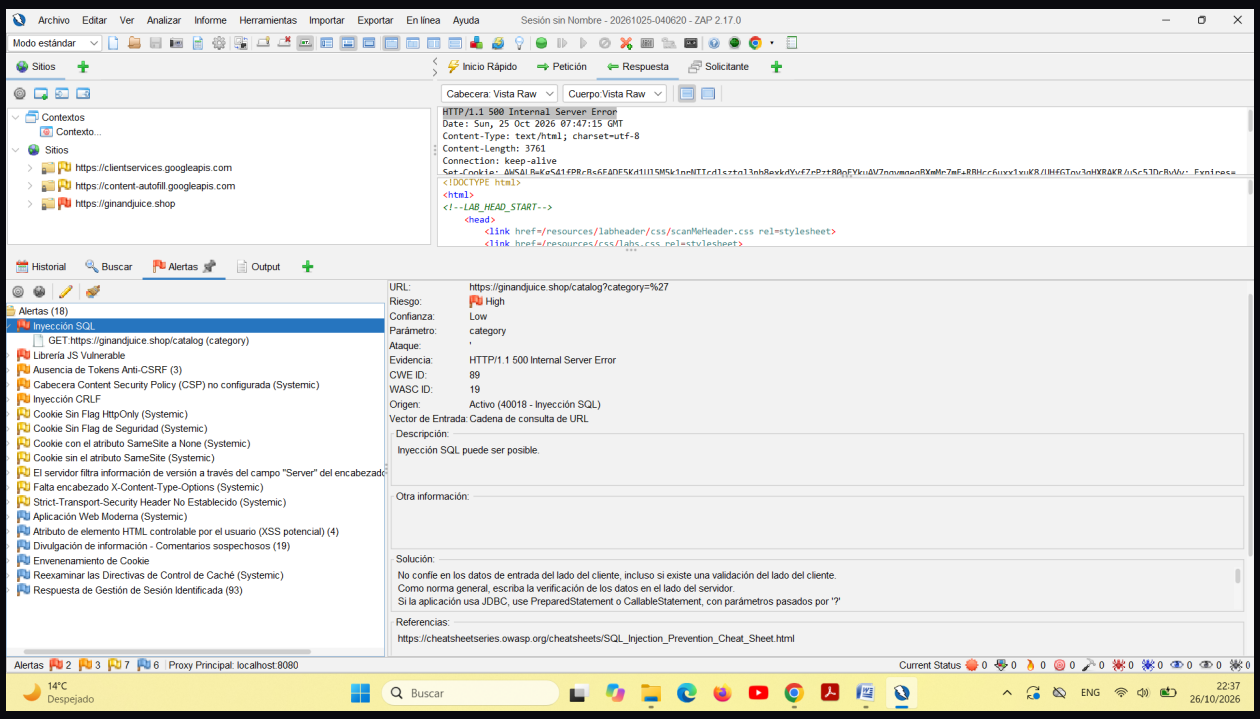
**Evidencia:**




**Explicación:** el servidor envía una cookie sin el atributo HttpOnly, por lo que puede leerse desde JavaScript. Si un atacante lograra ejecutar código en el navegador (por ejemplo, con XSS), podría robar esa cookie y, si es de sesión, suplantar al usuario. Se corrige agregando HttpOnly (y también Secure y SameSite) al crear la cookie.

## 4. Documenta un hallazgo principal

Elegí de las 3 alertas: Nombre de la alerta Inyección SQL.




| N.º | Campo | Detalle |
|---|---|---|
| 1 | Nombre de la alerta | Inyección SQL |
| 2 | Nivel de riesgo | Alto (confianza: baja) |
| 3 | URL o componente | https://ginandjuice.shop/catalog?category=%27 |
| 4 | Parámetro afectado | category |
| 5 | Evidencia | Al enviar una comilla simple ( ' ) en el parámetro, el servidor respondió HTTP 500 Internal Server Error. CWE-89, WASC-19. |
| 6 | Descripción técnica | El valor del parámetro category se inserta en una consulta SQL sin validar. Una comilla simple rompe la sintaxis de la consulta y provoca un error del servidor, señal de que la entrada llega directo a la base de datos. Detectado por el escaneo activo (regla 40018). |
| 7 | Impacto potencial | Un atacante podría modificar la consulta para leer datos de otras tablas (usuarios, contraseñas), alterar o borrar información, o incluso evadir la autenticación. |
| 8 | Recomendación | No confiar en los datos del lado del cliente. Usar consultas parametrizadas (PreparedStatement / CallableStatement en JDBC), validar y sanitizar la entrada en el servidor, y aplicar el principio de menor privilegio en la base de datos. |

## 5. Conclusiones

Documentá 3 elementos.

### ¿Qué aprendí?

Aprendí a usar OWASP ZAP para evaluar la seguridad de una aplicación web: configurar el proxy, explorar el sitio, ejecutar el Spider, el AJAX Spider y el Escaneo Activo, y leer las alertas. ZAP me mostró 18 tipos de vulnerabilidades ordenadas por riesgo, cada una con su URL, parámetro y evidencia.

### ¿Qué fue lo más retador?

Lo más difícil al principio fue conectar el navegador con ZAP y entender la diferencia entre el Spider, el AJAX Spider y el Escaneo Activo. También me costó interpretar tantas alertas y elegir cuáles documentar según su gravedad. Lo resolví siguiendo la guía y las descripciones que da ZAP.

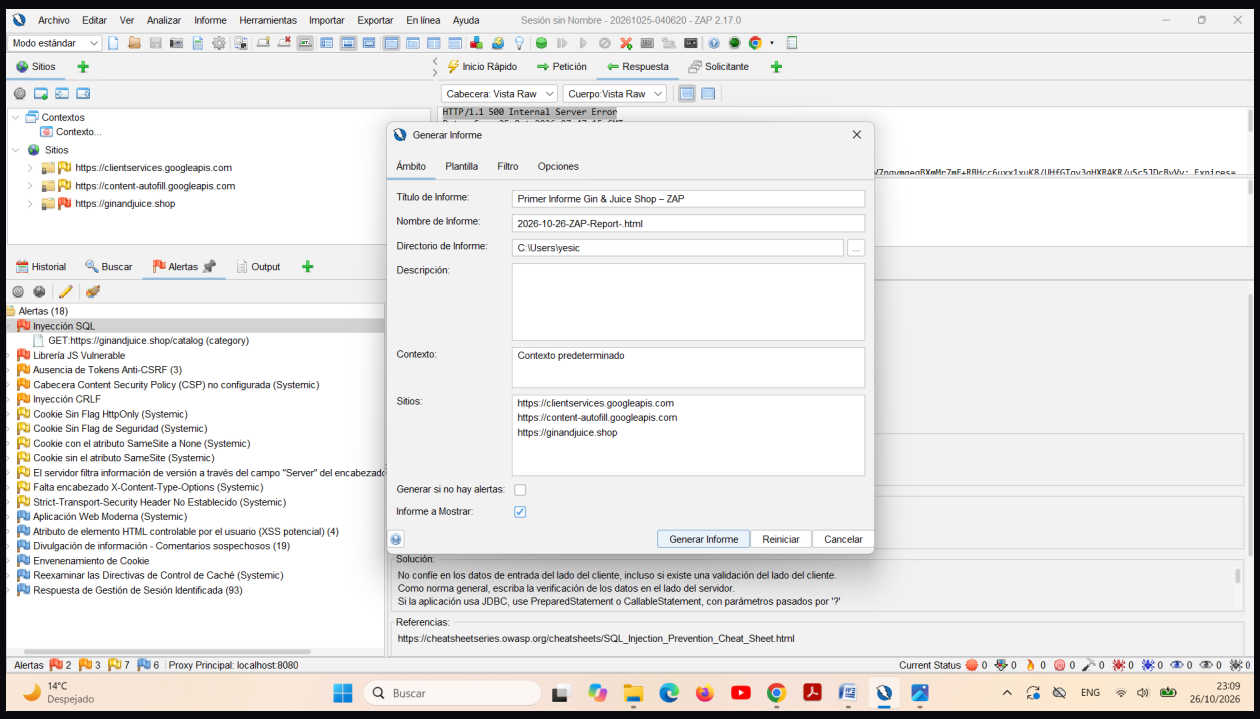
### ¿Qué revisaría a continuación?

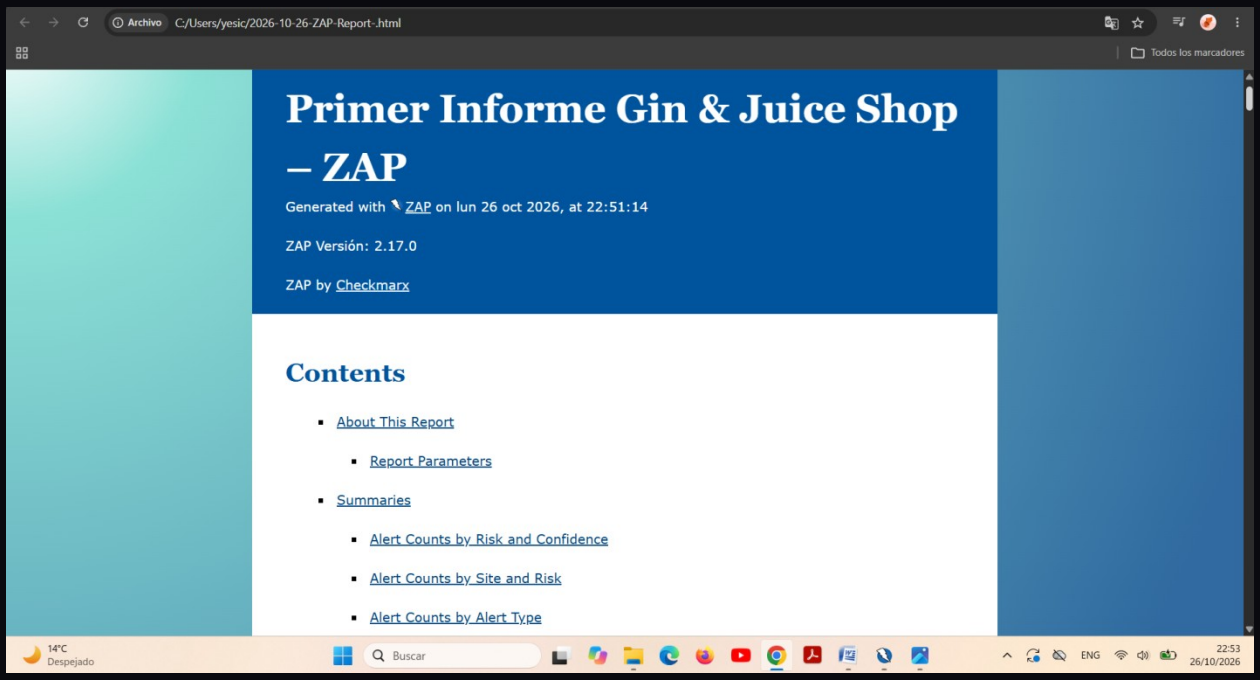
Confirmaría manualmente la Inyección SQL probando payloads en el parámetro category, revisaría la librería AngularJS 1.7.7 desactualizada, y probaría el login y el buscador en busca de XSS.

### Reporte final de ZAP

Generá el reporte desde Report:

file:///C:/Users/yesic/2026-10-26-ZAP-Report-.html




| Nivel de riesgo | Alerta | URL | Parámetro | Evidencia |
|---|---|---|---|---|
| Alto | Inyección SQL | ginandjuice.shop/catalog | category | Error HTTP 500 al enviar una comilla simple |
| Medio | CSP no | Varias | — | Falta la cabecera |

| Nivel de riesgo | Alerta | URL | Parámetro | Evidencia |
|---|---|---|---|---|
| | configurada | URLs | | Content-Security-Policy |
| Bajo | Cookie sin flag HttpOnly | Varias URLs | AWSALB | Cookie accesible desde JavaScript |